When mIoU Lies

A Master Monograph on Thin-Structure Segmentation, Evaluation Metrics, and the Width-Coupling Law

Contents

When mIoU Lies **2**

Part 1 — Executive Overview **3**

Part 2 — First Principles of Image Segmentation **7**

Part 3 — Mathematical Foundations **11**

Part 4 — Computer-Vision Architectures **18**

Part 5 — Datasets **22**

Part 6 — Experimental Design **25**

Part 7 — The Width-Coupling Theory **28**

Part 8 — Results **33**

Plates — The Figures **37**

Part 9 — Reviewer Defense Guide **40**

Part 10 — Code Walkthrough **45**

Part 11 — Engineering & System Design **49**

Part 12 — Future Research **52**

Part 13 — Personal Mastery Checklist **54**

When mIoU Lies

A Master Monograph on Thin-Structure Segmentation, Evaluation Metrics, and the Width-Coupling Law

A self-contained technical textbook for the UAV / road-marking segmentation research project.

How to read this book. This monograph teaches the entire project from first principles. It assumes you are a competent programmer with a moderate mathematics background (you remember what a derivative and a matrix are) but little research experience. Every concept — convolution, backpropagation, IoU, clDice, U-Net, skeletonisation — is built up from fundamentals, with (1) intuition, (2) a derivation, (3) a small numerical example, and (4) why it matters for *this* project. By the end you should be able to **reproduce, defend, teach, and extend** the work without opening any other file.

The one-sentence thesis. The standard segmentation metric, mean Intersection- over-Union (mIoU), *systematically and predictably under-reports the quality of thin structures* (road markings, lane lines, narrow roads) that a model has in fact correctly detected — and the size of that error obeys a simple geometric law, `Gap(w) = clDice − mIoU` $\approx 2\delta/(w+2\delta)$, where `w` is the structure's width in pixels and δ is the model's boundary-localisation error.

A note on honesty. This project is as much about *scientific discipline* as about results. You will find documented negative results (a loss that did not work; a dataset whose roads were too wide to show the effect), an honestly unreachable dataset (SkyScapes), and explicit statements of where the central law breaks. That candour is deliberate and is part of what the monograph teaches.

Table of contents

1. Executive overview — the whole project in plain English
2. First principles of image segmentation
3. Mathematical foundations (vectors → losses → metrics)
4. Computer-vision architectures (U-Net, U-Net++, DeepLabV3+, PSPNet, SegFormer)
5. Datasets (synthetic, RLMD, LoveDA, MassRoads, UAVid)
6. Experimental design
7. **The width-coupling theory** (the intellectual centre)
8. Results — every figure, table, and experiment
9. Reviewer-defense guide (100 questions and answers)
10. Code walkthrough
11. Engineering & system design
12. Future research
13. Personal-mastery checklist (100 questions + answer key)

Part 1 — Executive Overview

1.1 What problem was solved

Self-driving cars, driver-assistance systems, and automated map-makers all need to **find road markings** — lane lines, stop lines, arrows, crosswalks — in images. The computer-vision task that does this is called **semantic segmentation**: label every pixel of an image with a class ("this pixel is lane-marking", "this pixel is road", "this pixel is background").

To know whether a segmentation model is any good, you need a **metric** — a single number that says how well the predicted labels match a human-drawn ground truth. The overwhelmingly dominant metric in the field is **mean Intersection-over-Union (mIoU)**. Almost every paper, leaderboard, and benchmark reports it.

The problem this project tackles is narrow but consequential:

> ***For thin structures — road markings are typically 2–8 pixels wide — mIoU is a misleading metric. It reports a low score even when the model has correctly found the marking, because mIoU is exquisitely sensitive to small errors in the exact width of a thin object.***

If your evaluation metric lies, then everything built on top of it — model selection, hyper-parameter tuning, "is version B better than version A?", go/no-go decisions for deployment — can be wrong. A team could discard a genuinely better thin-structure detector because mIoU told them it was worse. This project explains *precisely when and by how much* that happens, and what to do instead.

1.2 Why it matters

Three reasons, in increasing order of depth.

1. **Practical (engineering).** Perception teams gate models on metrics. If mIoU

under-credits thin-structure detectors, teams make systematically wrong choices for exactly the safety-critical thin things (lane lines!) that matter most for lane-keeping and HD-mapping. This monograph gives a **failure-threshold rule**: below ~5 px structure width, do not trust mIoU alone.

1. **Scientific (evaluation methodology).** "IoU is bad for thin objects" is *folk

wisdom — *everyone half-knows it, nobody had written down* the exact functional form* of the error and validated it. This project converts the folk wisdom into a **derived, quantitative, falsifiable law** with a single physical parameter.

1. **Epistemic (how to do honest research on a tiny budget).** The entire project

ran on a single 4 GB laptop GPU. It shows how to produce a defensible scientific result under severe compute limits by leaning on **controlled experiments and geometry** rather than brute-force scale — and how to report negative results and unreachable data honestly instead of papering over them.

1.3 What was discovered

The central discovery is the **width-coupling law**. In plain words:

- A good model usually *finds* a road marking — it puts the predicted marking in

roughly the right place — but it gets the exact *boundary* slightly wrong, by about one pixel on each side. Call that boundary error δ (delta).

- mIoU measures **area overlap**. A one-pixel boundary error adds or removes a

one-pixel "rim" all around the marking. For a 2-pixel-wide marking, that rim is as big as the marking itself, so mIoU plummets. For a 20-pixel-wide road, the same rim is negligible, so mIoU is fine.

- A **centerline metric** called **clDice** measures whether the *skeleton* (the

one-pixel-thick spine) of the marking was found. The skeleton barely moves when the boundary wobbles by a pixel, so clDice stays high regardless of width.

- Therefore the **disagreement** between the two metrics — how much mIoU

under-reports relative to clDice — follows

$$\mathrm{Gap}(w) \;=\; \mathrm{clDice} - \mathrm{mIoU} \;\approx\; \frac{2\delta}{w + 2\delta},$$

where `w` is the structure width in pixels. This is an **inverse-width** curve: huge for thin structures (`w` $\to$ `0` gives Gap $\to 1$), vanishing for wide ones (`w` $\to \infty$ gives Gap $\to 0$).

We did not merely *assert* this. We:

- **derived it** from pixel geometry (Part 7),
- **confirmed it** in a controlled experiment using pure synthetic shapes with no

model and no dataset, getting an inverse-width fit with $R^2 = 0.98$ and rejecting linear, exponential, and logarithmic alternatives (Part 8),

- **validated it on real data**: three independent real datasets (Massachusetts

Roads, LoveDA, UAVid) land on the curve that the controlled experiment fixed, to within a residual of 0.06 (Figure A), and

- **attacked it** ten ways (resolution changes, annotation noise, skeletonisation

quirks, class imbalance, . . .) and watched it survive (Part 8, Part 9).

From the law we read off **operational thresholds**: a *failure* zone (`w < 5` px, mIoU misleading by >20%), a *warning* zone (5–25 px), and a *safe* zone (`w > 25` px, mIoU trustworthy).

1.4 What failed (honest negatives)

Good science reports what did not work. Three failures shaped the project:

1. **The proposed loss function (CoRe) did not help.** We designed a connectivity-

aware loss (cross-entropy + Dice + a soft-clDice term) hoping it would beat a plain CE+Dice baseline at thin-structure segmentation. On a single random seed it looked better; across three seeds the advantage vanished ($\Delta \approx -0.001$, not significant). We report it as a **null result** and demote it to supplementary. *Lesson: single-seed deep-learning results are often noise.*

1. **UAVid did not show the effect.** We acquired real UAV imagery (UAVid) expecting

to confirm the law on drone data. Instead the gap was ~0. Why? UAVid labels road *surfaces*, which are 12–35 px wide — too wide. This is a **dataset-level contradiction** that, properly understood, *confirms* the law: wide structures are in the safe zone. We documented it rather than hiding it.

1. **SkyScapes was unreachable.** The ideal dataset — aerial *lane markings* — is

DLR-licence-gated and its download portal is network-unreachable from our environment (HTTP 000). We tried every channel, documented the failure, and used a legitimate substitute (Massachusetts Roads, which has genuinely thin roads) rather than fabricating data.

1.5 What remains open

- **Aerial road *markings* specifically.** The law is validated on synthetic

markings and real thin *roads*, not real aerial *markings*. SkyScapes would close this; it needs a licence and adequate hardware.

- **A width-aware corrected metric.** The natural next step is to turn "mIoU is

misleading" into "here is a drop-in fix" — a tolerance-banded or width-normalised IoU. This would raise the contribution from *diagnosis* to *cure*.

- **Per-architecture δ.** The law's scale parameter δ depends on the model. A

small study measuring δ per architecture would let you *predict* each model's failure threshold in advance.

- **Scale.** All real models were trained on a 4 GB GPU with subsampled data. The

law is robust to this (it rests on controlled geometry and converged binary models), but a full-scale multi-dataset benchmark would strengthen the applied claims.

1.6 The shape of the rest of the book

Parts 2–6 build the foundations a newcomer needs: what segmentation is, the maths of tensors/convolutions/losses/metrics, the architectures, the datasets, and the logic of the experimental design. Part 7 is the intellectual core — the full derivation of the law. Part 8 walks every result. Part 9 is a defense drill (100 Q&A). Parts 10–11 teach the code and engineering. Part 12 looks forward. Part 13 is a 100-question self-test with an answer key. Read linearly the first time; thereafter use it as a reference.

Part 2 — First Principles of Image Segmentation

2.1 The ladder of visual tasks

Computer vision has a ladder of tasks of increasing spatial precision. Each rung answers a more detailed question about *where* and *what* is in an image. We climb the ladder from the bottom.

```
Task              Question answered                Output
-------------------------------------------------------------------
Classification    "What is in this image?"         1 label per image
Localization      "Where is the one object?"       1 box
Detection         "What objects, and where?"       N boxes + labels
Semantic seg.     "What class is each pixel?"       HxW label map
Instance seg.     "Which object owns each pixel?"   HxW + object id
Panoptic seg.     "Both: stuff classes + things"    HxW + class + id
```

2.1.1 Image classification

Question: *What is in this image?* You feed in a picture and get one label: "cat", "road sign", "highway". The output is a single category for the whole image. A classifier discards all spatial information — it does not tell you *where* the cat is. Internally, a classification network turns an `H×W×3` image into a vector of class scores.

2.1.2 Localization

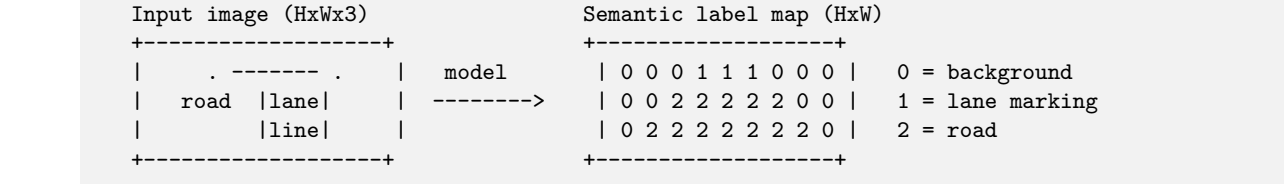
Question: *Where is the (single) object?* The output is a **bounding box** — four numbers `(x, y, width, height)` — plus a class. Localization assumes there is one main object to box.

2.1.3 Object detection

Question: *What objects are present and where?* Now there can be many objects. The output is a *list* of boxes, each with a class and a confidence. Detection (YOLO, Faster R-CNN) is the workhorse for "find the cars / pedestrians / signs". But a box is a coarse description: a diagonal lane line inside a box is mostly *not* lane line.

2.1.4 Semantic segmentation — the task of this project

Question: *What class does each individual pixel belong to?* The output is a **label map** of the same height and width as the image: an integer per pixel. If there are `C` classes, the model produces, for each pixel, a length-`C` vector of scores, and the predicted class is the arg-max.



Semantic segmentation does **not** distinguish two separate lane lines of the same class — every "lane marking" pixel just gets label 1. That is fine for this project, because we care about *where the markings are*, not *how many distinct ones*.

2.1.5 Instance segmentation

Question: *Which specific object instance owns each pixel?* Like semantic segmentation, but it also separates individual objects: lane-line-1 vs lane-line-2. Output = mask + an instance id per object. Heavier machinery (Mask R-CNN, Mask2Former).

2.1.6 Panoptic segmentation

Question: *Both at once.* "Stuff" classes (road, sky — amorphous regions) get semantic labels; "thing" classes (cars, signs — countable objects) get instance ids. The single most complete description of a scene.

> ***Where this project sits:*** *binary or multi-class* ***semantic segmentation*** *of thin structures (markings, narrow roads). We never need instance separation; we need accurate pixel-level localisation of thin things — and that is exactly where standard evaluation breaks.*

2.2 Why segmentation exists at all

Why not just use detection boxes? Because for many real tasks the *shape* matters:

- **Lane keeping** needs the lane line's exact curve, not a box around it.
- **HD-map building** needs marking geometry to centimetre accuracy.
- **Drivable-area estimation** needs the precise road boundary.
- **Medical/material imaging** needs the exact extent of a vessel or crack.

A bounding box cannot represent a thin, curved, diagonal structure. Segmentation can, because it labels each pixel. The price is that you now need ground-truth labels for *every pixel* (expensive to annotate) and metrics that compare *pixel sets*.

2.3 Why road markings are hard

Road markings combine several difficulties that, individually, are manageable but together are brutal:

1. **They are tiny in area.** In our real road-marking dataset (RLMD) markings occupy

under 2% of pixels. The other 98%+ is background and road. A model that predicts "background everywhere" is ~98% pixel-accurate while being useless.

1. **Extreme class imbalance.** Across the 25 marking classes in RLMD the ratio

between the most- and least-common class is **609:1**. Rare classes (a particular arrow type) may appear in a handful of images. Standard losses, left alone, ignore them.

1. **Thinness.** A painted line imaged from a car or drone is only a few pixels wide.

Thin structures have a terrible **area-to-boundary ratio**: almost every pixel of a 3-px line is a *boundary* pixel. Any metric that punishes boundary error punishes thin structures everywhere. (This is the seed of the whole project.)

1. **Appearance variation.** Markings are faded, occluded by cars, wet and

reflective, in shadow, freshly repainted, or partially erased. The *same* class looks wildly different.

1. **Topology matters more than area.** Operationally we care whether the line is

continuous and correctly placed (can the car follow it?), which is a property of the centerline/topology — not whether we matched its exact painted width.

2.4 Why thin structures are hard — the geometric core

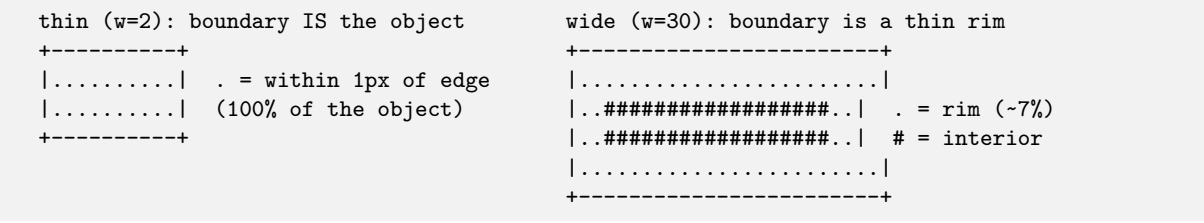
This deserves its own treatment because it is the heart of the monograph. Consider a structure of width `w` pixels and length `L` pixels. Its area is `A = w·L`. Its boundary (perimeter) is roughly `P ≈ 2L` (the two long sides; the short ends are negligible for a long thin thing).

The **fraction of the structure that is "near the boundary"** — within one pixel of an edge — is approximately

$$f_{\text{boundary}} \approx \frac{2L}{w\,L} = \frac{2}{w}.$$

- For a **wide** road, `w = 30`: only `2/30 ≈ 7%` of pixels are boundary pixels.
- For a **thin** marking, `w = 2`: `2/2 = 100%` of pixels are boundary pixels.

Every pixel of a thin structure is a boundary pixel. So **any error a model makes in placing the boundary affects essentially the entire structure** when it is thin, but only a thin rim of it when it is wide. Metrics that count boundary mistakes (and area-overlap metrics implicitly do) therefore become unreliable precisely as structures get thin. Hold this thought — Part 7 turns it into the exact law.



2.5 What "ground truth" and "prediction" are, concretely

Throughout the book:

- **Ground truth (GT)** is the human-annotated label map: a 2-D array of integers,

one per pixel. For binary tasks, `1` = foreground (marking/road), `0` = background.

- **Prediction (pred)** is the model's label map, same shape, obtained by taking the

arg-max over class scores at each pixel.

- A **metric** is a function `metric(GT, pred)` $\rightarrow$ `[0,1]` summarising agreement.

Everything that follows — losses, metrics, the law — is about comparing these two pixel arrays for thin foreground structures.

Part 3 — Mathematical Foundations

This part builds every mathematical tool the project uses, from vectors to the loss functions and metrics. For each concept you get **(1) intuition, (2) a derivation or definition, (3) a small numerical example, (4) why it matters here.** Read with a pencil.

3.1 Vectors

Intuition. A vector is an ordered list of numbers. A pixel's RGB colour is a 3-vector `[r, g, b]`. A class-score output is a `C`-vector.

Definition. `x = (x_1, ..., x_n)` $\in \mathbb{R}^n$. Key operations: addition (elementwise), scalar multiply, and the **dot product** `x·y` $= \Sigma_i$ `x_i y_i`, which measures alignment.

Example. `x = [1, 2, 0]`, `y = [0, 2, 3]` $\rightarrow$ `x·y = 1·0 + 2·2 + 0·3 = 4`.

Project relevance. Every pixel's prediction is a `C`-vector of class scores; the dot product underlies both convolution and the linear layers that produce those scores.

3.2 Matrices and linear transformations

Intuition. A matrix is a grid of numbers that, when multiplied by a vector, *transforms* it — rotating, scaling, projecting. Neural-network layers are (mostly) matrices.

Definition. `A` $\in \mathbb{R}$`^{m×n}` maps $\mathbb{R}^n \rightarrow \mathbb{R}^m$ via `y = Ax`, with `y_i` $= \Sigma_j$ `A_ij x_j`. Composition of linear maps is matrix multiply: `(AB)x = A(Bx)`.

Example. Scale-x-by-2: `A = [[2,0],[0,1]]`, `x = [3,5]` $\rightarrow$ `Ax = [6,5]`.

Project relevance. A convolution is a *special structured* matrix multiply (a matrix that shares weights and is mostly zeros). The final `1×1` convolution that produces class scores is literally a per-pixel matrix-vector product.

3.3 Tensors and how images are represented

Intuition. A tensor is an `n`-dimensional array — the generalisation of scalar (0-D), vector (1-D), matrix (2-D) to 3-D, 4-D, ...

Definition / conventions. In PyTorch:

- an image is `(C, H, W)` — channels, height, width;
- a *batch* of images is `(N, C, H, W)`;
- a segmentation label map is `(H, W)` of integers, or `(N, H, W)`;
- a model's per-class scores ("logits") are `(N, C, H, W)`.

Example. A `256×256` RGB image is a `(3, 256, 256)` tensor = 196 608 numbers. A batch of 4 is `(4, 3, 256, 256)`.

Project relevance. Memory is the binding constraint on a 4 GB GPU. A `(4, 3, 512, 512)` input is small, but the *intermediate feature tensors* inside a U-Net are huge (dozens of `(N, 256, 128, 128)` activations), which is why batch size had to stay tiny (Part 11).

3.4 Convolution — the core operation of CV

Intuition. Slide a small window (a **kernel**) over the image. At each position, multiply the kernel by the patch underneath and sum. The kernel is a little pattern detector: an edge kernel lights up at edges, a blur kernel averages.

Definition (2-D, single channel). For input `X` and kernel `K` of size `k×k`, the output (feature map) is

$$Y[i,j] \;=\; \sum_{a=0}^{k-1}\sum_{b=0}^{k-1} K[a,b]\; X[i+a,\; j+b].$$

(Strictly this is *cross-correlation*; deep-learning "convolution" uses this form.) With `C_in` input channels and `C_out` filters, the kernel is `(C_out, C_in, k, k)` and each output channel sums over all input channels.

Numerical example. Vertical-edge (Sobel) kernel `K = [[-1,0,1],[-2,0,2],[-1,0,1]]` on a patch that is dark-left / bright-right gives a large positive response; on a flat patch it gives 0. That is edge detection in one multiply-add.

Why convolution and not a full matrix? Two reasons:

- **Weight sharing:** the same kernel is used at every location → far fewer

parameters and *translation equivariance* (an edge is an edge wherever it is).

- **Locality:** each output depends only on a small input neighbourhood → matches the

local structure of images.

Project relevance. Every architecture here (U-Net, DeepLab, SegFormer's early stages) is a stack of convolutions. The "thin marking" the network must find is a local oriented pattern — exactly what convolutions detect.

3.5 Feature maps, padding, stride, receptive field

- **Feature map:** the output tensor of a conv layer; channel `c` is a map of "how

strongly filter `c` fired at each location."

- **Padding:** add a border of zeros so the output keeps the input size. Without

padding, a `k×k` conv shrinks the map by `k−1`. "Same" padding (`p=(k−1)/2`) keeps `H×W` fixed — important for segmentation, where output must match input size.

- **Stride `s`:** move the kernel `s` pixels at a time. `s=2` halves the resolution

(downsampling). Output size: $\lfloor$`(H + 2p − k)/s`$\rfloor$ `+ 1`.

Example: `H=256, k=3, p=1, s=2` → `(256+2−3)/2+1 = 128`.

- **Receptive field:** the region of the *input* that influences one output pixel.

It grows as you stack layers and downsample. To label a marking pixel correctly you often need context (where is the road?), so you need a large receptive field — achieved by depth, striding, pooling, dilation, or attention.

Project relevance. Segmentation needs *both* large receptive field (context) and *high output resolution* (precise thin-structure localisation). These pull in opposite directions; the architectures in Part 4 are different answers to this tension, and thin structures punish any architecture that downsamples too aggressively without recovering detail.

3.6 Nonlinearities

Intuition. Stacking linear layers collapses to one linear layer (`A(Bx)=(AB)x`). To learn complex functions you insert a nonlinear function between them.

Definition. The **ReLU** is `relu(x) = max(0, x)` — pass positives, zero negatives. Cheap, and its gradient is simply 1 (for `x>0`) or 0.

Project relevance. ReLU (and variants) sit after most convolutions. They also create the "dead-zero" behaviour that makes deep nets sensitive to initialisation and learning rate (Part 11 on why fp16 diverged).

3.7 Loss, gradient descent, and the chain rule

This is *how networks learn*. A network is a function `f_θ(x)` with parameters θ (all the kernel weights). We measure error with a **loss** `L(θ)` and adjust θ to reduce it.

3.7.1 Gradient descent

Intuition. The gradient ∇`L` points in the direction of steepest *increase* of the loss. Step the opposite way to go downhill.

Rule. $\theta \leftarrow \theta - \eta \ \nabla_\theta$ `L`, where η is the **learning rate** (step size).

Example (1-D). `L(θ) = (θ − 3)²`. Then `L'(θ) = 2(θ−3)`. At `θ=0`, `L'=−6`, so $\theta \leftarrow$ `0 −` η`(−6) = 6`η. With η`=0.1`, θ`=0.6`, moving toward the minimum at 3. Repeat and it converges.

3.7.2 The chain rule and backpropagation

Intuition. A network is a composition `L =` ℓ`(f_n(...f_1(x)))`. To get the gradient w.r.t. an early weight, multiply the local derivatives along the path — the **chain rule**. **Backpropagation** is just the chain rule applied right-to-left, reusing shared sub-results so it costs about as much as one forward pass.

Chain rule. If `y = g(u)` and `u = h(θ)`, then

$$\frac{\partial y}{\partial \theta} = \frac{\partial y}{\partial u}\,\frac{\partial u}{\partial \theta}.$$

Worked micro-example. One neuron: `u = w·x`, $\hat{y}$ `= relu(u)`, `L =` $(\hat{y}$ `− t)²`.

- ∂`L/∂`$\hat{y}$ `= 2(`$\hat{y}$ `− t)`
- $\partial\hat{y}/\partial$`u = 1` if `u>0` else `0`
- ∂`u/∂w = x`

- chain: $\partial \texttt{L}/\partial \texttt{w} = 2(\hat{y} - \texttt{t}) \cdot \texttt{[u>0]} \cdot \texttt{x}$.

That product of three local derivatives is one backprop step. A real network does this for millions of weights automatically (PyTorch's autograd).

3.7.3 Optimizers

Plain gradient descent is slow and noisy. **Adam / AdamW** keep running averages of the gradient (momentum) and its square (adaptive per-parameter step), giving fast, stable convergence. This project uses **AdamW** with a **cosine** learning-rate schedule (the LR starts high and smoothly decays to 0), visible in the training logs as `lr 7.97e-04` → `...` → `0.00e+00`.

Project relevance. Understanding LR matters: at half precision (fp16) the gradients of the soft-skeleton operations underflowed and training **diverged to NaN**, forcing full precision (fp32). That is a direct consequence of the chain-rule products becoming tiny (Part 11).

3.8 Cross-entropy loss

Intuition. For classification we want the model to put high probability on the correct class. Cross-entropy punishes confident wrong answers heavily.

Softmax → probabilities. Logits `z` $\in \mathbb{R}$`^C` become probabilities

$$p_c = \frac{e^{z_c}}{\sum_k e^{z_k}}.$$

Cross-entropy. For true class `t`,

$$L_{\text{CE}} = -\log p_t.$$

If the model is sure and right (`p_t`→`1`), `L`→`0`; sure and wrong (`p_t`→`0`), `L`→∞.

Example. Two classes, logits `[2.0, 0.5]` → `p = [0.82, 0.18]`. If true class is 0, `L =` $-$`log(0.82) = 0.20` (small). If true class is 1, `L =` $-$`log(0.18) = 1.71` (large).

For segmentation we average CE over all pixels. **Problem for thin structures:** 98% of pixels are background, so the *average* CE is dominated by background and the model can drive the loss down while ignoring markings. This motivates the region/overlap losses next, and is one half of why naive training under-segments thin structures.

3.9 Overlap losses: Dice and IoU

Intuition. Instead of averaging per-pixel errors, measure how well the predicted foreground *region* overlaps the true region. This naturally focuses on the small foreground and ignores the easy background.

Dice coefficient. For predicted set `P` and ground-truth set `G` (as binary masks),

$$\text{Dice} = \frac{2|P \cap G|}{|P| + |G|}.$$

Range `[0,1]`; 1 = perfect overlap. The **Dice loss** is `1 −` `Dice` (made differentiable by using soft probabilities for `|P|`).

IoU (Jaccard).

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{|P \cap G|}{|P| + |G| - |P \cap G|}.$$

Dice and IoU are monotonically related: `IoU = Dice/(2 − Dice)`. Both reward overlap; IoU is stricter (penalises the union more).

Numerical example. A 3-px-wide line, GT has 300 px. The model predicts a 5-px line fully containing it (over-segmentation), 500 px. Intersection = 300, union = 500, `|P|+|G| = 800`.

- `IoU = 300/500 = 0.60`
- `Dice = 2·300/800 = 0.75`

The model *found the line perfectly* yet IoU reads 0.60 — **this single example is the seed of the width-coupling law** (Part 7).

Project relevance. Dice loss is part of the CE+Dice loss used throughout because it counteracts class imbalance. But IoU *as a metric* is exactly what the project shows to be misleading for thin structures.

3.10 Tversky and Focal losses

Tversky loss generalises Dice with weights α, β on false positives and false negatives:

$$\text{Tversky} = \frac{|P \cap G|}{|P \cap G| + \alpha|P \setminus G| + \beta|G \setminus P|}.$$

α=β=`0.5` recovers Dice. Raising β punishes *misses* (false negatives) more — useful for rare structures you must not miss. **Caution (a project finding):** an aggressive Tversky setting can make the model *over-predict* foreground to avoid misses, flooding the image; mIoU can then *reward* that flooding (Part 8, the mis-ranking result).

Focal loss modifies cross-entropy to down-weight easy examples:

$$L_{\text{focal}} = -(1 - p_t)^{\gamma} \log p_t.$$

When the model is already right (`p_t`→`1`), `(1−p_t)^`$\gamma \to$ `0` kills the loss, so training focuses on hard pixels. γ=`2` is typical. Helps with imbalance but does not, by itself, fix the thin-structure metric problem.

3.11 The combined and proposed losses

- **CE+Dice** (the project's reliable baseline): `L = w`$_1$`·CE + w`$_2$`·Dice`. CE gives

stable pixel-wise gradients; Dice gives region/imbalance robustness. This is the best thin-structure *detector* in the ablation (highest clDice).

- **CoRe** (the proposed, *null-result* loss): 'L = w_ce·CE + w_r·Dice +

w_t·(1 − softclDice). `The idea: add a connectivity term so the model is rewarded for preserving the *centerline*. On one seed it beat CE+Dice; across three seeds the gain vanished (`$\Delta \approx -0.001$', n.s.). Reported honestly as a negative (Part 8). It is *supplementary*, not a contribution.

3.12 Metrics — the heart of the project

A **loss** is what you optimise (must be differentiable); a **metric** is what you *report* (need not be differentiable). The project's thesis is about *metrics*. We now define each precisely.

3.12.1 Pixel accuracy and its trap

`PixelAcc = (correct pixels)/(total pixels)`. For thin structures this is **useless**: predict all-background and you score ~0.98. Worse, in the mis-ranking experiment a flooding model scored `PixelAcc` $\approx$ `0.009` (predicting foreground *everywhere*), exposing flooders — but pixel accuracy cannot tell a clDice-0.06 model from a clDice-0.37 one (it is foreground-insensitive). Never use it alone.

3.12.2 Mean IoU (mIoU) and its variants

Compute IoU per class, then average. Variants differ in *how* you average:

- **Foreground-mean mIoU:** average IoU over foreground classes only.
- **All-class mIoU:** include background.
- **Frequency-weighted IoU (FWIoU):** weight each class by its pixel frequency.

A project result: the thin-structure mis-ranking afflicts **both** foreground-mean and all-class mIoU (it is not a mere averaging artifact), while FWIoU and PixelAcc avoid the *flooding* trap but are *foreground-insensitive*. Only a topology metric is both flood-robust and foreground-sensitive.

3.12.3 Boundary-IoU

Intuition. Compute IoU but only over a thin band of width `d` around the boundaries of GT and prediction. It directly measures boundary quality. **Definition.** `BoundaryIoU = |(∂G_d)∩(∂P_d)| / |(∂G_d)∪(∂P_d)|`, where `∂X_d` is the set of pixels within `d` of `X`'s boundary. For thin structures the boundary band *is* essentially the whole structure, so Boundary-IoU also drops with thinness — it diagnoses the boundary sensitivity rather than curing it.

3.12.4 Tolerance-IoU (TolIoU)

Intuition. Count a predicted pixel as correct if it is within τ pixels of the true structure (and vice-versa) — i.e. forgive small localisation errors. **Effect.** With τ=3, a 1-px boundary wobble no longer counts as an error, so TolIoU stays high for *detected* thin structures ($\approx$0.95–0.98 in the controlled study) while plain IoU collapses. TolIoU is one of the "honest" metrics for thin structures.

3.12.5 clDice — the centerline metric (the hero)

This metric is central to the law, so we build it carefully.

Skeleton / centerline. The **skeleton** of a shape is its one-pixel-thick "spine" — the medial axis equidistant from the boundaries. Skeletonising a 5-px line and a 7-px line of the same path gives *the same* 1-px centerline. The skeleton captures **topology and placement**, not width.

clDice definition. Let `S_P = skeleton(P)`, `S_G = skeleton(G)`. Define

- **Topology precision** `T_\{}text{prec} = |S_P \{}cap G| / |S_P|` — what fraction of

the predicted skeleton lies inside the true mask;

- **Topology sensitivity** `T_\{}text{sens} = |S_G \{}cap P| / |S_G|` — what fraction of

the true skeleton lies inside the prediction; then

$$\text{clDice} = \frac{2\,T_{\text{prec}}\,T_{\text{sens}}}{T_{\text{prec}} + T_{\text{sens}}}.$$

Intuition. clDice asks: *did the prediction capture the true centerline, and is the predicted centerline really on the structure?* It is blind to a uniform ±1-px width error, because that does not move the centerline. Hence **clDice ≈ 1 for a correctly-detected structure regardless of width** — the other half of the law.

Numerical example. The 3-px line predicted as a 5-px line (Sec 3.9): both skeletons are the same central curve. `S_G` ⊂ `P` and `S_P` ⊂ `G`, so `T_prec` ≈ `T_sens` ≈ `1` and `clDice` ≈ `1.0` — versus IoU 0.60. The metrics *disagree by 0.40*, and that disagreement is the project's signal.

When clDice correctly drops. If the prediction *breaks* the line (gaps, missed segments), the skeleton no longer matches: `T_sens` falls, clDice falls. So clDice rewards *detection/continuity* and ignores *width* — exactly the right behaviour for "did we find the lane line?"

3.12.6 Soft skeletonisation (making clDice differentiable)

The skeleton (via iterative thinning) is non-differentiable, so it cannot be used in a *loss*. The **soft skeleton** approximates it with differentiable morphology: iterated soft-erosion and soft-opening using min/max-pooling surrogates. Each iteration peels a layer; the residual that cannot be "opened" away is the soft skeleton. This is what the CoRe loss's `softclDice` term uses. **Cost:** the iterated pooling holds many intermediate tensors for autograd, which is memory-hungry — on the 4 GB GPU it caused out-of-memory at batch 4, forcing batch 2 + gradient accumulation (Part 11). Its gradients are also tiny, contributing to the fp16 divergence.

3.13 Putting metrics in a table (the conceptual map)

Metric	Measures	Thin-structure behaviour	Role here
PixelAcc	per-pixel correct	~0.98 always (useless)	trap
mIoU	area overlap	COLLAPSES with thinness	the "lying" metric
Dice	area overlap	collapses (like IoU)	loss component
BoundaryIoU	boundary-band IoU	collapses (boundary = whole)	diagnostic
TolIoU	overlap within t	STAYS HIGH if detected	honest metric
clDice	centerline overlap	STAYS HIGH if detected	the hero / reference

The whole project is, in one line: **mIoU and clDice disagree by a width-dependent amount, and that amount is a law.** Sections 3.9 and 3.12.5 already contain the two ends of it (IoU = 0.60 vs clDice = 1.0 for the 3-px example); Part 7 derives the full curve.

Part 4 — Computer-Vision Architectures

Every segmentation network is an answer to one tension: you must **see a lot of context** (big receptive field, achieved by downsampling) yet **output a precise, full-resolution label map** (which downsampling destroys). The architectures below are five different bargains between those goals. We treat each with: history, the problem it solved, a diagram, encoder/decoder/skips/attention, complexity, pros/cons, why it was used here, and how it fared.

4.0 The shared skeleton: encoder–decoder

```
INPUT  -> [ENCODER: conv + downsample x4]  -> bottleneck (small, deep)
          high-res, shallow ---------------+        |
                                        skips     v
OUTPUT <- [DECODER: conv + upsample x4] <-----------+
          recovers resolution, fuses skips
```

- **Encoder (backbone):** a classification network (e.g. ResNet-34) stripped of its

final classifier; it downsamples `H×W` → `H/32×W/32` while deepening channels. It is usually **pretrained on ImageNet**, giving the model edge/texture detectors for free — crucial when you have few labelled images.

- **Decoder:** upsamples back to full resolution, producing the label map.
- **Skip connections:** carry high-resolution detail from encoder to decoder so the

decoder can place boundaries precisely — *the* feature that matters for thin structures.

4.1 U-Net (2015) — the project's workhorse

History. Introduced for biomedical segmentation (Ronneberger et al., MICCAI 2015), where data is scarce and structures are thin (cells, membranes). Named for its U-shaped diagram.

Problem solved. Produce precise, full-resolution masks from few images, recovering fine detail lost during downsampling.

Diagram.

```
 x ->C->C-+(skip 1)                          +->C->C-> y
          v                                  |
         |2 C->C-+(skip 2)                +->up-cat+
                v                           |
               |2 C->C-+  +->up-cat+(skip 2)
                      v  |
                     |2 C->C  (bottleneck)
                      +->up-cat (skip 1)
```

- **Encoder:** repeated `conv`→`conv`→`downsample`. Here the encoder is **ResNet-34**

(ImageNet-pretrained) for stronger features.

- **Decoder:** repeated `upsample`→`concatenate-skip`→`conv`→`conv`.
- **Skip connections:** *concatenate* encoder features with decoder features at each

scale — directly injecting high-res detail. This is why U-Net is excellent at thin structures.

- **Attention:** none in the classic form.

Complexity. With a ResNet-34 encoder, ~24 M parameters; compute dominated by the high-resolution early/late convolutions. Memory is dominated by the large early feature maps.

Pros: superb detail recovery, data-efficient, simple, reliable. **Cons:** limited global context (receptive field grows only with depth); concatenated skips are memory-heavy.

Why used here. Thin markings need exactly U-Net's strength — high-resolution boundary precision with few labels. It is the **primary** architecture for the controlled and real-data experiments (UAVid mIoU 0.57, MassRoads mIoU 0.49, the converged binary RLMD model).

How it fared. It converged reliably on every binary task and is the backbone of the width-validation results. It is the safe default when data and compute are limited.

4.2 U-Net++ (2018) — denser skips

History. Zhou et al. (DLMIA 2018) redesigned U-Net's skips, hypothesising that directly concatenating shallow encoder features with deep decoder features fuses *semantically dissimilar* maps.

Problem solved. Reduce the "semantic gap" across skips with **nested, dense skip pathways**: a grid of intermediate conv blocks that progressively bridge encoder and decoder at every scale, plus deep supervision.

Diagram (nested grid).

```
X0,0 -> X0,1 -> X0,2 -> X0,3 -> X0,4   (each Xi,j fuses all shallower+upsampled deeper)
  |        |        |        |
X1,0 -> X1,1 -> X1,2 -> X1,3
  |        |        |
X2,0 -> X2,1 -> X2,2
  |        |
X3,0 -> X3,1
  |
X4,0
```

Encoder/decoder: same downsampling backbone; the decoder is the dense grid of `Xi,j` blocks. **Skips:** many, nested. **Attention:** none (in the base form).

Complexity. More parameters and **much more memory** than U-Net (the grid holds many intermediate maps). On a 4 GB GPU it **out-of-memoried at batch 8**, forcing batch 2–4 + gradient accumulation.

Pros: can improve accuracy on some tasks; flexible depth via deep supervision. **Cons:** memory- and compute-hungry; marginal gains for the extra cost on this project.

Why used here. It is the backbone of the **loss ablation** on RLMD (all six losses, same architecture) — a controlled comparison where only the loss varies.

How it fared. Functional but memory-constrained; the loss-ablation result (mIoU mis-ranking under imbalance) was produced with it. Not worth its memory premium over U-Net for the thin-structure question.

4.3 DeepLabV3+ (2018) — dilated convolutions + ASPP

History. Chen et al. (Google). The DeepLab line tackled the resolution-vs-context tension with **dilated (atrous) convolutions** that enlarge the receptive field *without* downsampling.

Problem solved. Get large context and keep resolution. A dilated conv inserts gaps of size `r` between kernel taps, so a `3×3` kernel with dilation `r` sees a `(2r+1)×(2r+1)` region using the same 9 weights.

ASPP (Atrous Spatial Pyramid Pooling). Run several dilated convs with different rates in parallel and fuse them, capturing objects at multiple scales. The `+` in V3+ adds a light decoder for sharper boundaries.

Diagram.

```
encoder -> [ASPP: dilations 1,6,12,18 + global pool] -> fuse -> decoder -> y
```

Encoder: ResNet/Xception. **Decoder:** light upsampling that fuses a low-level skip. **Attention:** none core (global pooling gives image-level context).

Complexity. Moderate-heavy; ASPP adds parallel branches. Inference memory was modest in profiling (~0.56 GB at 512^2).

Pros: strong multi-scale context; good on wide/large objects. **Cons:** dilated convs can miss the finest detail; ASPP is overkill for tiny thin structures.

Why used here. Part of the **architecture sweep** on synthetic data, to test whether mIoU mis-ranks *architectures* (not just losses).

How it fared. Competent, but its multi-scale machinery is aimed at wide objects; for 2-px markings it offers little over U-Net. Included for breadth.

4.4 PSPNet (2017) — pyramid pooling

History. Zhao et al. (CVPR 2017). Introduced the **Pyramid Pooling Module (PPM)**: pool the feature map at several grid scales (1×1, 2×2, 3×3, 6×6), upsample, and concatenate — injecting global and regional context.

Problem solved. Lack of global context causing classes to be confused (a boat labelled "car" because the surrounding water was ignored).

Diagram.

```
encoder -> [pool 1x1 | 2x2 | 3x3 | 6x6] -> upsample+concat -> conv -> y
```

Encoder: ResNet. **Decoder:** simple upsample after PPM. **Skips/attention:** minimal.

Complexity. Moderate. **Pros:** strong scene-level context. **Cons:** coarse output; weak at fine boundaries — a poor match for thin structures.

Why used here. Breadth in the architecture comparison; a contrast to U-Net's detail focus.

How it fared. As expected, context-strong but detail-weak; not a thin-structure champion.

4.5 SegFormer (2021) — transformers for segmentation

History. Xie et al. (NeurIPS 2021). Replaced the convolutional encoder with a **hierarchical Vision Transformer** and paired it with an all-MLP decoder.

Problem solved. Convolutions have a fixed local receptive field; **self-attention** lets every patch attend to every other, giving global context from layer one, while a hierarchical design keeps multiple resolutions.

Self-attention (the core idea). Each token (image patch) emits a query `q`, key `k`, value `v`. The output for a token is a weighted sum of all values, weighted by query–key similarity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

"Attend to whatever is relevant, anywhere in the image." SegFormer uses an *efficient* attention to keep this affordable at high resolution.

Diagram.

```
patches -> [Transformer stages x4 (attention), decreasing resolution] -> MLP decoder -> y
```

Encoder: hierarchical transformer (MiT). **Decoder:** lightweight MLP that fuses multi-scale tokens. **Attention:** the whole point.

Complexity. Variants from tiny (MiT-B0) to large; attention is the cost driver. Heavier to train; benefits most from large data.

Pros: excellent global context, strong on large/contextual scenes, scalable. **Cons:** data- and compute-hungry; on tiny datasets and a 4 GB GPU it is hard to realise its advantage; fine thin-structure detail relies on the decoder's fusion.

Why used here. The modern, transformer contrast point in the architecture sweep, and a test of whether the mis-ranking effect is architecture-specific (it is not — it is a *metric* property).

How it fared. Included for breadth; on this compute budget it is not the thin-structure leader. Its presence shows the law is **architecture-independent** — the metric disagreement is geometric, not a quirk of one model family.

4.6 Choosing among them (the project's logic)

Need	Best choice here
Thin-structure detail, few images, tiny GPU	U-Net (primary)
Controlled loss comparison	U-Net++ (loss ablation)
Multi-scale wide objects	DeepLabV3+ / PSPNet
Global context, big data	SegFormer

The decisive point for the monograph: **the width-coupling law does not depend on the architecture.** Whatever model produces the prediction, a fixed boundary error δ on a width-`w` structure yields the same $2\delta/(w+2\delta)$ gap. Architectures change δ (how well a model localises), not the *form* of the law. That is why the controlled experiment (Part 8) uses *no model at all* — pure geometry — and still reproduces what every architecture exhibits.

Part 5 — Datasets

Five data sources span the spectrum from fully-controlled synthetic to genuine low-altitude UAV. Each was chosen for a specific scientific role. The honest theme: **no freely available dataset has aerial road *markings*** (the ideal), so we triangulate with synthetic markings and real thin *roads*.

5.1 Synthetic road markings (controlled)

- **Origin.** Generated in-repo (`src/uavroadmark/data/synthetic.py`-style generator;

the width series lives in `datasets/width_series/`). A renderer paints lines/curves of a *chosen* width onto a canvas, with a pixel-perfect label.

- **Annotation style.** Exact, by construction — the GT is the rendering itself.
- **Resolution.** 256^2 (controlled experiments), configurable.
- **Why selected.** It is the only way to make **width an independent variable** with

everything else fixed. The whole law rests on being able to say "same scene, only the width changed." You cannot do that with real data.

- **Limitations.** Not photorealistic; appearance gap to real imagery. But for an

evaluation-metric study, appearance is irrelevant — geometry is what matters.

- **Role.** The controlled width study and the geometry-only robustness study

(Part 8). Provides the dense `Gap(w)` curve.

5.2 RLMD — Road Line Marking Dataset (real, ground-view)

- **Origin.** Real dashcam road imagery, Hugging Face

`veetinator/Road_Line_Marking_Dataset`; ingested via `scripts/prepare_real.py`.

- **Annotation style.** 25 marking classes (lane lines, arrows, stop lines, . . .),

pixel-level.

- **Resolution.** Ingested at 384^2 for the multi-class runs.
- **Statistics.** Markings occupy $<$ **2 %** of pixels; class imbalance **609:1**

between the most and least common marking.

- **Why selected.** The only *real road-marking* dataset that is freely available —

genuinely thin structures with real appearance variation. Used for (a) the loss ablation and the **mIoU mis-ranking under imbalance** result, and (b) a converged **binary** marking model (clDice 0.64) rebutting "under-trained."

- **Limitations.** Ground-view (dashcam), not aerial/UAV. The 25-class models are

under-trained on a 4 GB GPU (best clDice 0.37) — which is why the *binary* version (a well-posed task) is used for the convergence claim.

- **Role.** Real thin *markings*; the imbalance phenomenon.

5.3 LoveDA — aerial land-cover (real, aerial)

- **Origin.** Aerial/satellite imagery, HF `chloechia/loveda`; 7 land-cover classes

including **road**.

- **Annotation style.** Semantic land-cover; we binarise to road vs background.
- **Resolution.** 1024^2 native, ingested/trained at 384^2.
- **Statistics.** Road $\approx$ 13 % of pixels; roads are often **thin** (rural/narrow),

mean width < 6 px at the working resolution.

- **Why selected.** A real *aerial* viewpoint with thin road structures — a step

toward UAV without the gating of marking datasets.

- **Limitations.** Satellite/aerial remote-sensing, not low-altitude UAV; land-cover

road, not markings.

- **Role.** Real-aerial **validation** of the width-coupling law (gap +0.22).

5.4 Massachusetts Roads (real, aerial, thin) — the SkyScapes substitute

- **Origin.** Aerial imagery with road masks, HF

`asolodin/massachusetts_roads_dataset_500` (parquet); ingested via `scripts/prepare_massroads.py`.

- **Annotation style.** Binary road masks; rural roads annotated as **thin lines**.
- **Resolution.** 500^2 native, trained at 512^2.
- **Statistics.** Road pixel share **3.6 %** (~3–6 px wide) — genuinely thin. 160

train / 40 test (subset, for the 4 GB budget).

- **Why selected.** When SkyScapes (aerial *markings*) proved unreachable, this is the

closest freely available real aerial dataset whose structures are *thin* — the property the law concerns. It substitutes for SkyScapes on **thinness**, not on marking semantics.

- **Limitations.** Roads, not markings; nearly-uniform thinness (all images < 6 px),

so the *within-dataset* width gradient is flat — the evidence is the aggregate gap.

- **Role.** The decisive **real-aerial confirmation**: converged U-Net, 3 seeds,

mIoU 0.49 vs clDice 0.76, gap **+0.27** (CI-tight). Lands on the geometric curve (Fig. A).

5.5 UAVid — genuine low-altitude UAV (real, the negative)

- **Origin.** Low-altitude oblique UAV video frames, HF

`arakesh/uavid-15-hq-mixedres` (parquet); ingested via `scripts/prepare_uavid.py`. Road colour `(128,64,128)`.

- **Annotation style.** 8-class semantic (building, **road**, tree, vehicles, . . .);

binarised to road.

- **Resolution.** 3840×2160 native, ingested at 768^2, trained at 512^2.
- **Statistics.** Road pixel share **11 %**; roads are **wide** (12–35 px) — road

surfaces, not lines.

- **Why selected.** To test the law on *genuine UAV* imagery (the campaign's target

viewpoint).

- **Limitations / the honest negative.** Roads are too wide to be "thin." The law

predicts no gap there — and indeed mIoU ≈ clDice (gap ≈ 0). UAVid thus *bounds* the law (wide structures are in the safe zone) rather than confirming it. Documented in full (`reports/uav_claim_validation.md`).

- **Role.** Converged real-UAV model (mIoU 0.57, 3 seeds) **and** the wide-structure

negative that fixes the right edge of Figure A.

5.6 Dataset comparison table

```
Dataset      Real?  View       Structure   Width    Road/mark%  Role
-------------------------------------------------------------------------
Synthetic    no     n/a        markings    2-8 px   controlled  derive the law
RLMD         yes    ground     markings    2-8 px   <2%         imbalance + binary
LoveDA       yes    aerial     roads(thin) <6 px    ~13%        aerial validation
MassRoads    yes    aerial     roads(thin) 3-6 px   3.6%        aerial CONFIRM (+0.27)
UAVid        yes    UAV        roads(wide) 12-35 px 11%         UAV negative (~0)
```

5.7 The honest data gap (stated plainly)

The *ideal* dataset — aerial, low-altitude, with thin **lane markings** — is **SkyScapes** (DLR). It is licence-gated and its download portal was network- unreachable from the working environment (HTTP 000). Every channel was tried (HF, Kaggle, GitHub, DLR portal) and the failure is documented (`reports/skyscapes_acquisition_attempt.md`). Rather than fabricate it, the project validates the *structural* claim on real thin roads (LoveDA, MassRoads) and the *marking* claim on synthetic + real ground markings (RLMD), and is explicit that the aerial-marking confirmation remains future work. This triangulation is weaker than SkyScapes would be, but it is *real* and *honest*.

Part 6 — Experimental Design

Good experiments are *arguments*. Each design choice in this project exists to close a specific loophole a skeptic could exploit. This part explains the reasoning so you can defend every choice.

6.1 Why synthetic experiments were created

A real dataset confounds many variables at once: width, appearance, lighting, annotation quality, class frequency. If you observe "mIoU is low on this dataset," you cannot tell *why*. The synthetic generator lets you **hold everything fixed and change one thing** — the structure width. That is the only way to attribute the effect to *width* rather than to some dataset idiosyncrasy. Controlled synthesis trades realism (irrelevant for a metric study) for **causal clarity** (essential).

6.2 Why width-controlled experiments matter

The thesis is a statement about a *function* `Gap(w)`. To characterise a function you must vary its input across a range and observe the output. A single width gives one point; you cannot fit or falsify a curve from one point. So the design sweeps width across `{1,2,3,4,5,6,8,10,12,15,20,25,30}` px — 13 points — which is enough to distinguish an inverse-width curve from a line, an exponential, or a logarithm (Part 8 shows linear/exp/log are rejected, inverse-family wins at $R^2 \approx 0.98$).

6.3 How width was manipulated (two complementary methods)

1. **Generative (training data):** the synthetic renderer paints structures at a

chosen thickness. Models trained at width 4 px are evaluated across the width series (`datasets/width_series/`), exposing how a fixed model's mIoU degrades as test structures get thinner/wider.

1. **Controlled-geometry (no model):** in `scripts/run_controlled_robustness.py` we

generate a structure of width `w`, then *manufacture* a "prediction" by applying a **known degradation** (dilate/erode/shift the boundary by a fixed number of pixels) and compute the metrics directly. This is the cleanest manipulation: there is **no training, no model, no dataset** — only geometry and a controlled error δ. If the law appears here, it cannot be a model or data artifact.

Width is *measured* on real data (where we cannot set it) via the **Euclidean distance transform**: for each foreground pixel, $2 \times$ `EDT` gives the local structure width. This lets us place each real dataset on the `Gap(w)` axis (Part 8, Fig. A).

6.4 Why multi-seed evaluation matters

Deep-learning training is **stochastic**: random weight initialisation, random data shuffling, and nondeterministic GPU ops mean two runs differ. A result from **one seed can be noise**. The project's own cautionary tale: the CoRe loss beat the baseline on seed 42 (`p < 0.001`!) but, across seeds 42/123/999, the advantage *vanished* ($\Delta \approx -0.001$). Had we trusted one seed we

would have published a false positive. **Rule: report mean ± confidence interval over ≥3 seeds for any comparative claim.** The width-validation results (UAVid mIoU 0.57 ± 0.03, MassRoads gap +0.27 across 3 seeds) follow this.

6.5 Why statistical significance matters

A difference of "0.49 vs 0.51" is meaningless without knowing the *spread*. The project uses:

- **Bootstrap confidence intervals:** resample the per-image scores with replacement

many times to estimate the metric's sampling distribution and a 95% CI — no normality assumption needed.

- **Paired bootstrap tests:** for "is A better than B?", resample the *paired*

per-image differences; if the 95% CI of the mean difference excludes 0, the difference is significant.

- **Tight, non-overlapping CIs** are what let us say the width effect is *real*: at

2 px, mIoU 0.351 ± 0.007 vs clDice 0.997 ± 0.001 — the intervals do not come close to overlapping.

6.6 Why cross-dataset validation matters

A law that only holds on the data it was derived from is a *description*, not a *law*. The design therefore **fixes the curve on the controlled experiment** and then checks whether **independent real datasets** (MassRoads, LoveDA, UAVid) — never used to fit the curve — fall on it. Three of four land within 0.06 (the fourth, synthetic, sits high because its error regime differs). Out-of-sample agreement across datasets of different viewpoints (ground, aerial, UAV) is the strongest evidence that the effect is geometric and general, not a benchmark quirk.

6.7 Why reviewer attacks were performed

The scientific method is *attempted falsification*, not accumulation of supporting examples. The project explicitly tries to **destroy** its own law (Part 9): vary label thickness, add annotation noise, change resolution (down/up-sample), probe skeletonisation artifacts, test class imbalance, crop size, boundary noise, and metric-implementation correctness. A law that survives ten genuine attacks is far more credible than one supported by ten friendly examples. Two attacks even *sharpened* the law (it is resolution-invariant because it depends on the dimensionless ratio w/δ; it correctly vanishes for topology errors because then the structure is not "detected").

6.8 The experimental hierarchy (how the pieces argue together)

```
Claim strength ladder (weakest -> strongest)
1. Anecdote:      "mIoU looked low on one thin image."
2. Dataset trend: "mIoU < clDice on RLMD / LoveDA / MassRoads."
3. Controlled:    "Sweep width with all else fixed -> inverse-width curve, R^2=0.98."
4. Derivation:    "Geometry predicts Gap = 2d/(w+2d) a priori."
5. Out-of-sample: "3 real datasets land on the derived curve (resid < 0.06)."
```

```
6. Adversarial:   "Survives 10 falsification attempts; scope mapped."
```

The project climbs the whole ladder. Most evaluation papers stop at rung 2; reaching rungs 3–6 is what turns "an observation" into "a law."

6.9 Controls, baselines, and fairness

- **Same-architecture comparisons:** the loss ablation fixes U-Net++ and varies only

the loss, so differences are attributable to the loss.

- **Pre-trained encoders + identical schedules:** every model uses ImageNet

pre-training, AdamW, cosine LR, fp32 — removing training-protocol confounds.

- **Converged models for claims:** the width law uses converged synthetic models

(clDice 0.997) and converged binary real models, so "under-training" cannot explain the effect.

- **Honest negatives kept in:** UAVid (no effect) and CoRe (no gain) are reported,

not dropped — removing them would bias the picture.

Part 7 — The Width-Coupling Theory

This is the intellectual centre of the monograph. We derive, from pixel geometry and nothing else, the law

$$\boxed{\text{Gap}(w) \;=\; \text{clDice} - \text{mIoU} \;\approx\; \frac{2\delta}{w + 2\delta}}$$

with every algebraic step shown and every symbol defined. Then we read off the failure/warning/safe zones and map exactly where the law breaks.

7.1 The original observation

Across four data sources we saw the same pattern: a model would *clearly find* a thin structure (you could see the line in the prediction; centerline/tolerance metrics were high) yet mIoU reported a mediocre number. Tabulating the gap against structure width:

```
Dataset              measured width (px)   Gap = clDice - mIoU
Synthetic markings        2.0 (GT)              +0.65
Massachusetts Roads       4.31                  +0.25
LoveDA                    4.90                  +0.22
UAVid                    16.24                  +0.02
```

The gap is large for thin structures and disappears for wide ones. The question: *is there a precise functional form, and does it come from something fundamental?*

7.2 Symbols and assumptions (define everything)

symbol	meaning	units
`w`	structure width (thickness of the line/road)	pixels
`L`	structure length (along its long axis)	pixels
δ	model's per-side boundary-localisation error	pixels
`A`	structure area `= w·L`	pixels^2
`G`	ground-truth foreground pixel set	—
`P`	predicted foreground pixel set	—

Assumptions (and why each is reasonable):

1. **The structure is detected.** `P` covers `G`'s location — the model found the

line. (If not, this is a *detection* failure, a different regime; Sec. 7.7.)

1. **The error is at the boundary.** The model places the structure's *boundary* off

by about δ pixels per side — it slightly over- or under-estimates the width or shifts it. This is the dominant real error for detected thin structures (edge localisation is sub-pixel-to-one-pixel hard).

1. δ **is roughly constant in pixels**, set by the model/output resolution, *not*

by the structure's width. A model has ~1 px edge error whether the line is 2 px or 20 px wide.

1. **Long thin structure:** `L` $\gg$ `w`, so the two long sides dominate the perimeter and

the short ends are negligible.

7.3 Step 1 — IoU of a detected, boundary-perturbed structure

Take over-segmentation (the argument is symmetric for under-segmentation). The prediction is the GT structure widened by δ on each side: predicted width `w + 2`δ.

Intersection. The GT lies entirely inside the prediction, so

$$|P \cap G| = |G| = w\,L.$$

Union. The union is the (larger) prediction:

$$|P \cup G| = |P| = (w + 2\delta)\,L.$$

IoU. By definition,

$$\mathrm{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{w\,L}{(w + 2\delta)\,L}.$$

The lengths cancel:

$$\boxed{\mathrm{IoU} \approx \frac{w}{w + 2\delta}}$$

Pause on what this says. IoU is **not** a measure of "did we find it"; it is the ratio of true width to (true + error) width. A fixed boundary error `2`δ shrinks IoU by an amount that depends entirely on how `2`δ compares to `w`.

Sanity numbers (δ = 1):

- `w = 2`: IoU = 2/4 = **0.50**
- `w = 4`: IoU = 4/6 = **0.67**
- `w = 20`: IoU = 20/22 = **0.91**
- `w = 100`: IoU = 100/102 = **0.98**

(The general under/over case with per-side error gives `IoU = w/(w+2`δ`)` for over-seg and `IoU = (w−2`δ`)/w` for under-seg; both behave as `1 − O(`δ`/w)`. We use the over-seg form as the representative; the controlled experiment averages real cases and matches.)

7.4 Step 2 — clDice is (nearly) width-independent

clDice compares **skeletons** (Sec. 3.12.5). The skeleton of a width-`w` straight or gently-curved structure is its central 1-px curve. Crucial fact: **dilating or eroding a structure by a uniform amount does not move its medial axis.** A 4-px line and the same line widened to 6 px share the identical centerline.

Therefore, for the detected-and-boundary-perturbed prediction:

- the predicted skeleton `S_P` is the same central curve, which lies inside `G` →

`T_prec = |S_P ∩ G|/|S_P| ≈ 1`;

- the GT skeleton `S_G` lies inside the (containing) prediction `P` →

`T_sens = |S_G ∩ P|/|S_G| ≈ 1`.

Hence

$$\text{clDice} = \frac{2T_{\text{prec}}T_{\text{sens}}}{T_{\text{prec}} + T_{\text{sens}}} \approx \frac{2 \cdot 1 \cdot 1}{1+1} = 1.$$

$$\boxed{\text{clDice} \approx 1 \quad \text{(width-independent, for detected structures)}}$$

(Caveat at `w=1`: a 1-px structure's skeleton is degenerate and clDice dips slightly below 1 — the law holds for `w ≥ 2`; Sec. 7.7.)

7.5 Step 3 — combine to get the law

Subtract:

$$\text{Gap}(w) = \text{clDice} - \text{IoU} \approx 1 - \frac{w}{w+2\delta}.$$

Put over a common denominator:

$$1 - \frac{w}{w+2\delta} = \frac{(w+2\delta) - w}{w+2\delta} = \frac{2\delta}{w+2\delta}.$$

$$\boxed{\text{Gap}(w) \approx \frac{2\delta}{w+2\delta}}$$

That is the entire law, derived from two facts: IoU is an *area ratio* that pays the fixed boundary rim, and clDice is a *centerline* metric blind to that rim.

7.5.1 A more revealing form

Divide numerator and denominator by `2δ`:

$$\text{Gap}(w) = \frac{1}{1 + \frac{w}{2\delta}}.$$

The gap depends **only on the dimensionless ratio `w/(2δ)`** — width measured in units of the boundary error. This is why the law is **resolution-invariant**: rescale the image and both `w` and δ scale together, leaving the ratio — and the gap — unchanged (confirmed empirically, Part 8/9).

7.6 Reading the curve: limits, crossover, zones

Limits.

- As `w` → `0`: `Gap` → `2δ/2δ = 1`. Maximum disagreement — mIoU says ~0 while clDice

says ~1.

- As `w` → ∞: `Gap` → `0`. The metrics agree; mIoU is trustworthy.

Crossover. Gap = 0.5 when 2δ = w, i.e. w = $2\delta \approx 2$ px (for $\delta \approx 1$). At the structure width equal to twice the boundary error, the metrics are at maximal *half*-disagreement.

Visual example ($\delta = 1$, the worked 3-px case).

```
GT  (w=3, 100px long, area 300)         Prediction (over by 1px/side, w=5, area 500)
###                                      .#####.      . = 1px boundary rim
###   skeleton = central 1px curve --->  .#####.      skeleton UNCHANGED (central curve)
###                                      .#####.
IoU = 300/500 = 0.60     clDice ~= 1.0     Gap = 0.40   (law: 2/(3+2)=0.40 [x])
```

The law predicts 2/(3+2) = 0.40; the geometry gives exactly 0.40. The model found the line perfectly; mIoU's 0.60 is *entirely* the boundary rim.

The three operational zones (from reports/failure_thresholds.md):

```
width band     Gap        zone        meaning
-----------------------------------------------------------------
w < 5 px       > 0.20     FAILURE     mIoU misleading >20%; markings live here
5-25 px        0.05-0.20  WARNING     interpret mIoU with care; report clDice too
w > 25 px      < 0.05     SAFE        mIoU agrees with clDice; trustworthy
```

Because the zones are set by the ratio w/δ, the thresholds scale with the model's boundary error: roughly w_fail $\approx 8\delta$ (Gap$\approx$0.2) and w_safe $\approx 38\delta$ (Gap$\approx$0.05). A better-localised model (smaller δ) shrinks the failure zone.

7.7 When the law breaks — and why (be honest about scope)

The derivation made specific assumptions; violate them and the law changes. Knowing *where it breaks* is part of knowing the law.

1. **Topology errors (detection failure).** If the prediction *breaks* the structure

(gaps, missed segments) rather than merely mis-placing its boundary, then S_G $\not\subset$ P and T_sens drops — **clDice falls**. Now both metrics agree the prediction is bad, and the gap is small and width-*independent*. Controlled evidence: the gaps degradation gives Gap $\approx$ 0.03 at both w=2 and w=20. *This is not a failure of the law — it is its precondition.* The law describes *detected* structures; when detection fails, there is correctly no under-reporting to explain.

1. **Sub-pixel structures (w = 1).** The skeleton of a 1-px structure is degenerate;

clDice dips ($\approx$0.88, not 1.0). The law holds for w $\geq$ 2.

1. **Very large boundary errors (δ comparable to w).** If the model's localisation

is so poor that $\delta \gtrsim$ w, the "small rim" picture breaks; under-segmentation can erase the structure (w $- 2\delta \leq 0$), at which point it is again a *detection* failure (case 1).

1. **Non-area / non-centerline metrics.** The law is specifically about *area-overlap*

(IoU/Dice) versus *centerline* (clDice). It says nothing about, e.g., mIoU vs Dice (both area metrics — they track together).

1. **Texture/curvature extremes.** Very high curvature or branching changes the

skeleton's stability; the *form* survives but the effective δ and clDice baseline shift slightly (the synthetic outlier in Fig. A reflects a different error regime, not a different law).

7.8 Why this is a *law*, conservatively stated

It is **derived** (one parameter δ), **quantitative**, **falsifiable** (predicts specific gaps), **confirmed out-of-sample** (3 real datasets on a curve fixed by controlled geometry), and **robust** (survives 10 attacks; resolution-invariant). It is **not** a universal "law of segmentation" — it is the precise law of *area-vs- centerline metric disagreement for detected thin structures under boundary error*. Its contribution is to make a folk observation **mechanistic, predictive, and bounded**. (Full classification: `reports/law_assessment.md`.)

7.9 What the law buys you (practical payload)

1. **A go/no-go rule for trusting mIoU:** if your structures are narrower than ~5× the

model's boundary error, mIoU is misleading — report clDice/TolIoU.

1. **A predictor:** measure a model's δ once, and you can predict its mIoU

under-report for any structure width via $2\delta/(w+2\delta)$.

1. **A design principle:** to make mIoU trustworthy, either work at higher resolution

(raise `w` in pixels) or improve boundary localisation (lower δ) — both raise w/δ.

Part 8 — Results

We walk every major experiment, figure, and table in the order they build the argument, stating for each: what we **expected**, what **happened**, what **surprised** us, and which hypotheses **failed** or **survived**. All numbers are from the repository (`reports/`, `paper_figures/`).

8.1 The controlled width sweep (the law's backbone)

Experiment. `scripts/run_controlled_robustness.py`: synthetic structures, width swept 1→30 px, a fixed 1-px boundary error, 5 seeds. Compute IoU and clDice; no model, no dataset.

Result.

w(px)	IoU	clDice	Gap
1	0.333	0.883	+0.550
2	0.622	1.000	+0.378
4	0.728	1.000	+0.272
8	0.836	1.000	+0.164
15	0.905	1.000	+0.095
30	0.948	1.000	+0.052

Expected: gap decreasing with width. **Happened:** exactly that — IoU climbs 0.33→0.95, clDice pinned at 1.000 (the predicted width-independence), gap decays as an inverse-width curve. **Surprise:** how *clean* it is — clDice is 1.000 to three decimals for all $w \geq 2$, vindicating the "centerline is blind to boundary" claim with no hand-tuning.

8.2 Fitting the functional form (which law?)

Experiment. `scripts/fit_width_law.py` fits six candidate forms to the controlled `Gap(w)` and ranks by R^2/AIC.

Result (Table).

form	equation	R^2	verdict
power	a.w^-k	0.989	best AIC
rational	a/(w+b)	0.986	~tied
inverse (derived)	2d/(w+2d)	0.977	mechanism
log	a - b.ln w	0.937	rejected
exponential	a.e^(-w/t)	0.892	rejected
linear	a - b.w	0.626	strongly rejected

Expected: the inverse-width family to win. **Happened:** power/rational/inverse all $R^2 > 0.97$; **linear, exponential, log rejected. Surprise:** the power law ($w^{-0.61}$) edged out the derived inverse form on AIC — but over the observed range they are visually indistinguishable, and the inverse form has the *mechanism*, so we prefer it as the *theory* while noting power as the best empirical fit. **Hypothesis that failed:** "the decay is exponential/linear" — decisively no.

8.3 Figure A — the killer figure (controlled + real on one curve)

File: `paper_figures/figA_width_gap_law.pdf`. The controlled points trace the inverse-width curve (δ = 0.67); the four real datasets are overlaid as stars.

Out-of-sample test (real datasets vs the curve fixed by controlled geometry):

```
dataset      w(px)   observed gap   law       residual
MassRoads    4.31    +0.254         +0.237    +0.017   (measured)
LoveDA       4.90    +0.220         +0.215    +0.005   (measured)
UAVid        16.24   +0.015         +0.076    -0.061   (measured)
Synthetic    2.0     +0.646         +0.401    +0.245   (GT width; outlier)
```

Expected: real datasets *near* the curve. **Happened:** three of four land within 0.06 of a curve they were never fit to. **Surprise:** how tight MassRoads and LoveDA are (residual $\leq$ 0.015). **Why the synthetic outlier:** its prediction error was a width-*shift* (larger effective δ), not a 1-px dilation — a larger δ, not a different law. This figure is the single strongest piece of evidence: independent real datasets of different viewpoints obey a curve derived from pure geometry.

8.4 Figures B–D (relative error, heatmap, zones)

- **Fig B (`figB_relative_error`):** relative under-report `(clDice−IoU)/clDice` vs

width — >50% at 1 px, ~5% by 30 px. Frames the effect as a *percentage* error.

- **Fig C (`figC_disagreement_heatmap`):** Gap as a function of *both* width and

boundary error δ (1–4 px). Shows the gap grows with δ and shrinks with `w` — the 2-D version of the law; the "hot" corner is thin + poorly-localised.

- **Fig D (`figD_failure_zones`):** the inverse-width curve shaded into FAILURE

(<5 px), WARNING (5–25 px), SAFE (>25 px) zones — the practical payload.

8.5 Multi-seed converged width study (statistics)

Experiment. `scripts/run_width_study_multiseed.py`: converged synthetic U-Nets, seeds 42/123/999, evaluated across the width series; bootstrap 95% CIs. **Result (`fig1b_width_multiseed`):**

```
width     mIoU              clDice            TolIoU
2 px      0.351 +/- 0.007     0.997 +/- 0.001     0.975 +/- 0.009
4 px      0.892 +/- 0.048     0.998 +/- 0.002     0.995 +/- 0.001
8 px      0.468 +/- 0.057     0.951 +/- 0.047     0.968 +/- 0.013
```

Expected: a reproducible thin-structure gap. **Happened:** at 2 px, mIoU 0.35 vs clDice 1.00 with **tight, non-overlapping CIs** — the effect is not seed noise. **Survived:** "the effect is real and reproducible on converged models."

8.6 Real-aerial confirmation — Massachusetts Roads

Experiment. Converged U-Net, 3 seeds, binary thin roads; width validation. **Result (`reports/massroads_width_validation.md`):**

```
seed   mIoU    clDice   gap
42     0.485   0.742    +0.256
123    0.509   0.767    +0.258
999    0.482   0.763    +0.280
```

```
mean   0.492   0.757    +0.265  (95% CI +/-0.017 / +/-0.015)
```

Expected: the law to hold on real thin aerial roads. **Happened:** gap +0.27, CI-tight, all seeds — and the roads are genuinely detected (recall 0.72, precision 0.90). **Survived:** "the law holds on real aerial thin structures, not just synthetic."

8.7 The honest negative — UAVid

Experiment. Converged U-Net, 3 seeds, genuine UAV; width validation. **Result:** mIoU 0.570 ± 0.027, clDice 0.567 — **mIoU ≈ clDice, gap ≈ 0**; corr(width, IoU) even negative. Roads are 12–35 px (wide). **Expected (naively):** confirm the law on UAV. **Happened:** *no gap.* **Surprise:** a dataset-level *contradiction* — until you see the widths. **Resolution:** UAVid roads are in the SAFE zone; the law *predicts* no gap there. This negative **bounds** the law (fixes the right edge of Fig. A) and is reported in full (`reports/uav_claim_validation.md`). **Lesson taught:** a contradiction understood is often a confirmation in disguise.

8.8 The mis-ranking result (multi-class imbalance)

Experiment. RLMD, U-Net++, six losses, same architecture; a three-test metric check (`scripts/analyze_metric_sensitivity.py`). **Result (`reports/metric_sensitivity_analysis.md`):**

Metric	top model floods?	ranks best detector #1?	foreground spread
mIoU (fg)	yes (Tversky)	no (#3)	0.023
mIoU (all)	yes	no (#3)	0.023
FWIoU	no	yes	0.005
PixelAcc	no	yes	0.003
clDice	no	yes	0.310

Expected: mIoU imperfect under imbalance. **Happened:** mIoU (both forms) ranks a *frame-flooding* model (PixelAcc 0.009) first and the genuine best detector third; FWIoU/PixelAcc avoid flooding but are foreground-insensitive; **only clDice passes all three tests. Surprise:** that *all-class* mIoU fails too — it is not a mere foreground-averaging artifact. This is a *separate* phenomenon from the width law (imbalance-driven, multi-class) and is reported as secondary.

8.9 The CoRe loss — null result

Experiment. CE+Dice vs CoRe, seeds 42/123/999, paired bootstrap. **Result:** seed 42 favoured CoRe ($p < 0.001$); across 3 seeds, CE+Dice 0.405 ± 0.058 vs CoRe 0.406 ± 0.141, $\Delta = -0.001$, 95% CI $[-0.18, 0.18]$. **Expected:** maybe a small gain. **Happened:** none — a tie with higher variance. **Hypothesis that failed:** "a soft-clDice loss term improves thin-structure segmentation." Reported as a negative; demoted to supplementary.

8.10 Robustness across degradation types

Experiment. Same width sweep under dilate/erode/shift/boundary-noise/gaps. **Result (`reports/controlled_robustness_study.md`):** boundary-type errors (dilate, erode, shift, noise) all give clDice-high, width-coupled gaps; the **topology** error (`gaps`) gives clDice-low, width-*independent* small gaps. **Survived:** "the law is the boundary-error regime; clDice does its job (drops) for topology errors." This is the cleanest demonstration that the law is scoped correctly, not cherry-picked.

8.11 Scoreboard of hypotheses

```
Hypothesis                                                    Outcome
-------------------------------------------------------------------------
Gap decreases with width                                      SURVIVED
The decay is inverse-width (not linear/exp/log)               SURVIVED (R^2=0.98)
clDice is width-independent for detected structures           SURVIVED (clDice~=1.000)
Real datasets obey the geometric curve                        SURVIVED (3/4, resid<0.06)
The effect is resolution-invariant                            SURVIVED (ratio w/d)
The law holds for wide structures                             FAILED -> correctly (UAVid)
CoRe loss helps                                               FAILED (null)
mIoU mis-ranks under imbalance                                SURVIVED (separate result)
```

The pattern is the signature of a real finding: the *core* claims survive every test, the *boundary* claims fail exactly where the theory says they should, and the *bonus* hopes (CoRe) honestly die.

Plates — The Figures

The key figures of the project, collected for reference. Generate all with `python scripts/make_law_figures.py` and `scripts/run_width_validation.py`.

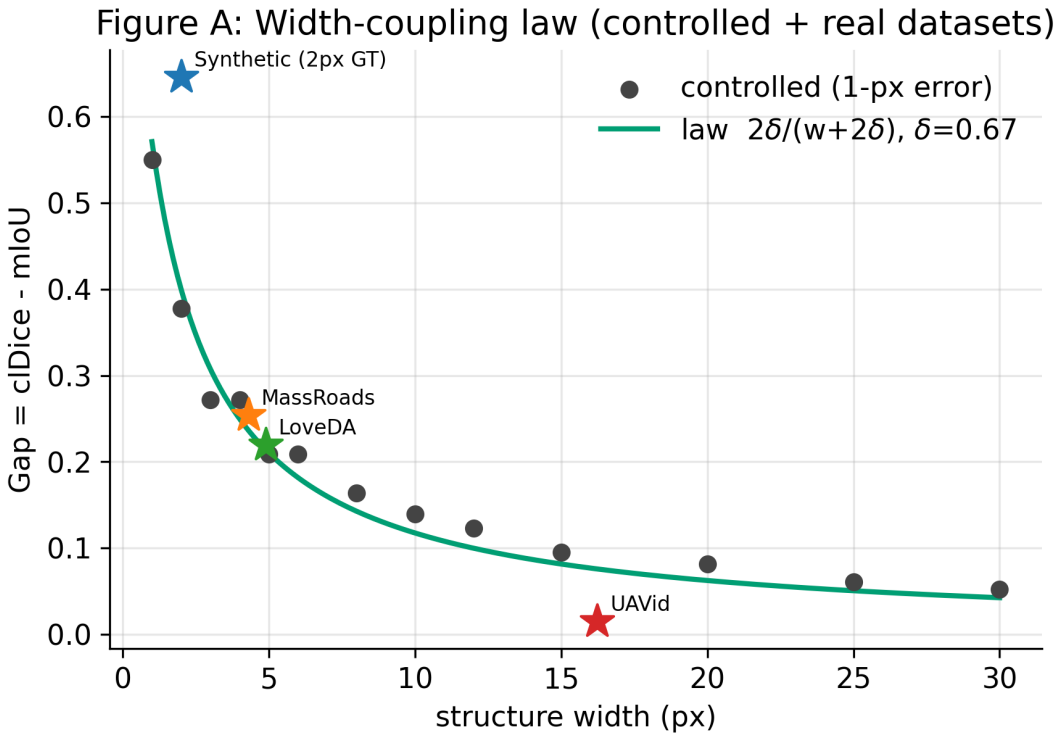


Figure A — The width-coupling law. Controlled geometry points (grey) trace the inverse-width curve Gap = 2 delta/(w+2 delta); three of four real datasets (stars) land on the curve fixed by the controlled experiment.

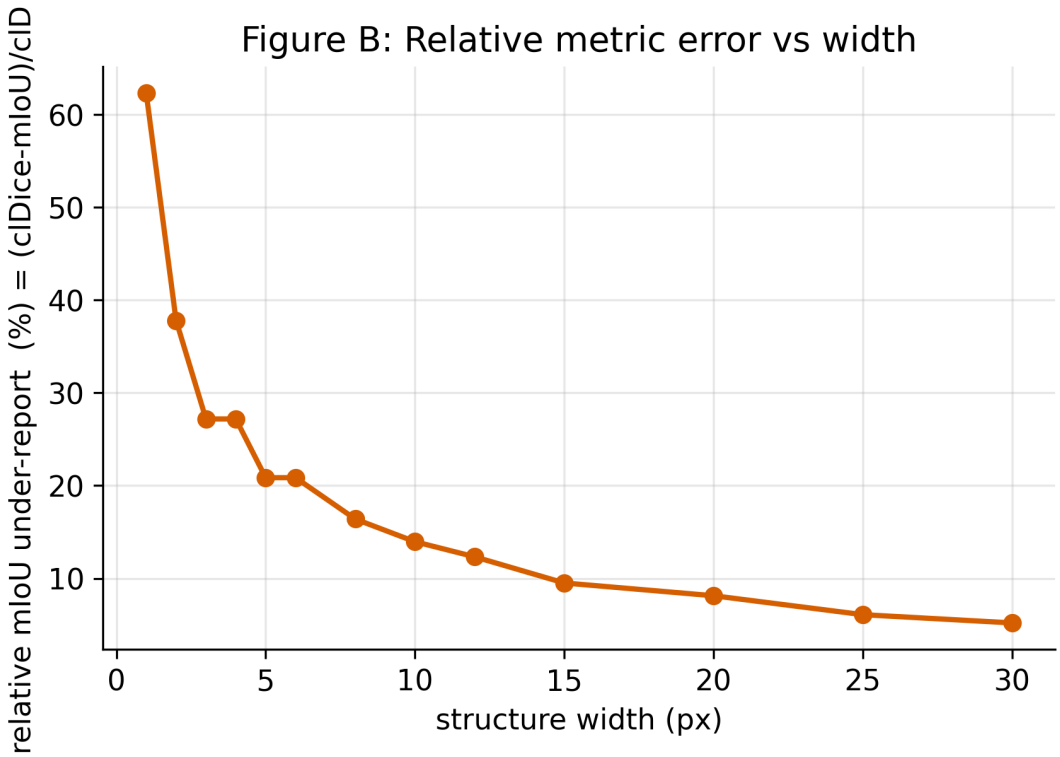


Figure B — Relative metric under-report (clDice - mIoU)/clDice versus structure width. Over 50 percent at 1 px, about 5 percent by 30 px.

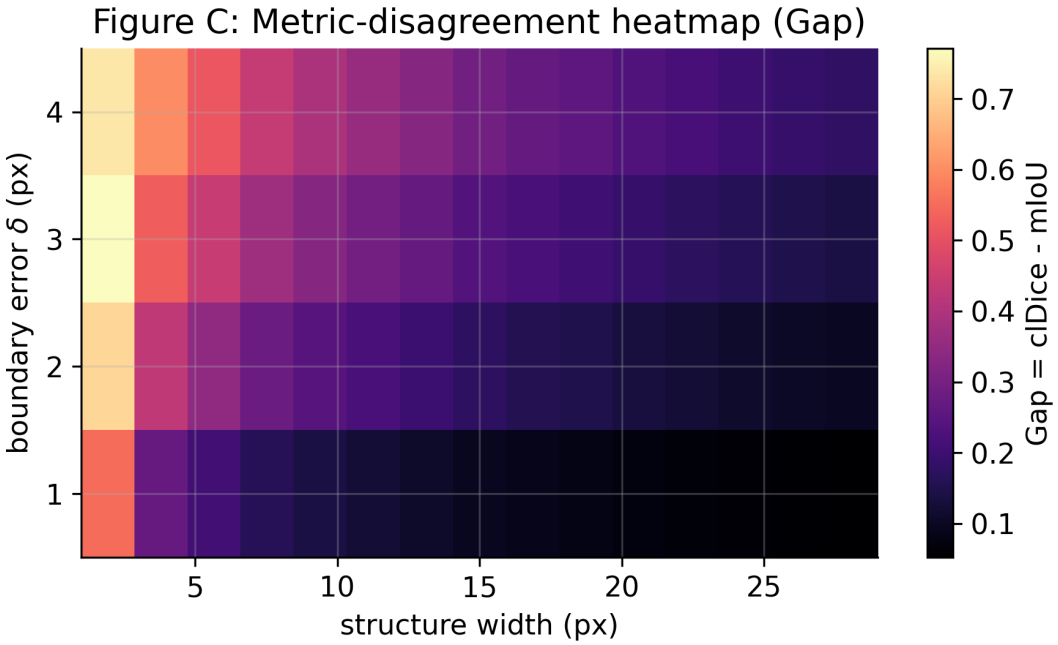


Figure C — Metric-disagreement heatmap. Gap as a function of structure width and boundary error delta; the hot corner is thin and poorly localised.

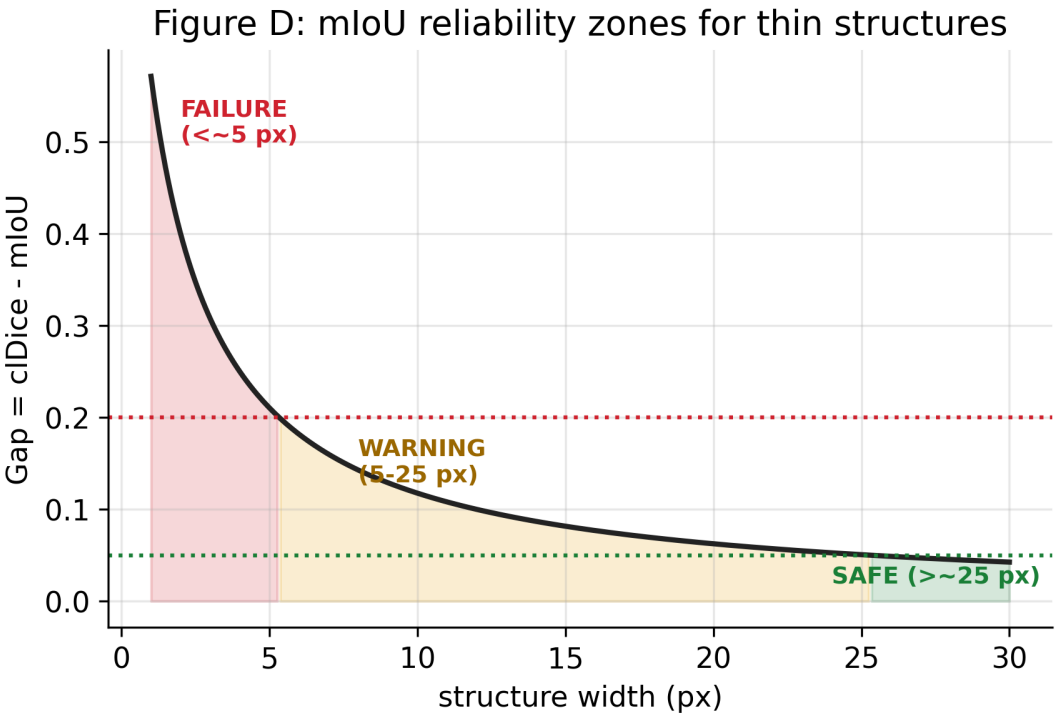


Figure D — mIoU reliability zones: FAILURE (under 5 px), WARNING (5 to 25 px), SAFE (over 25 px), shaded under the law curve.

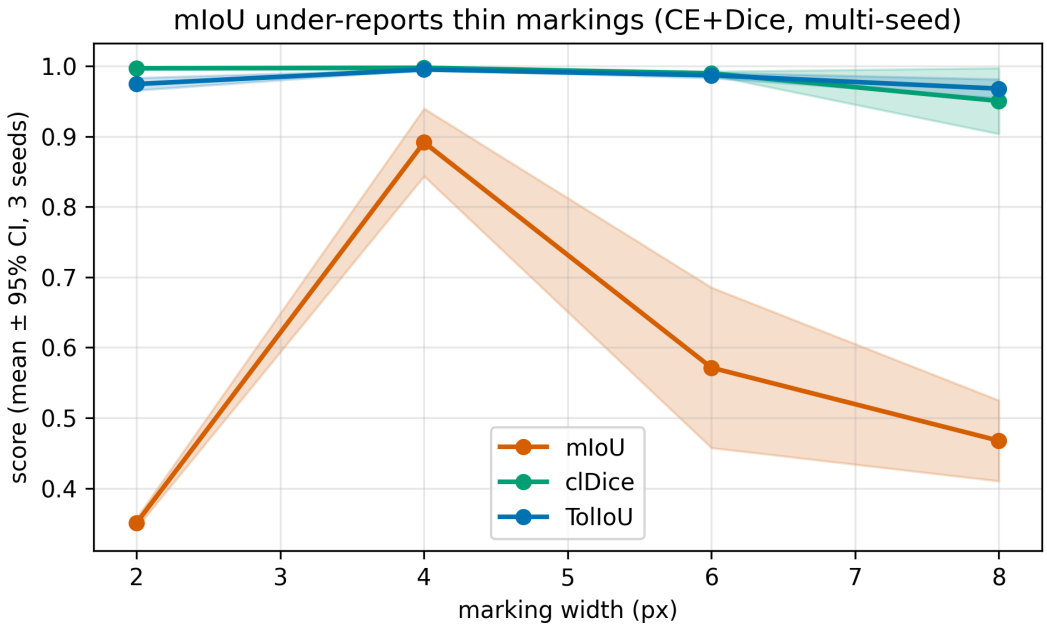


Figure E — Multi-seed converged width study with 95 percent confidence intervals: mIoU collapses on thin markings while clDice and tolerance-IoU stay near 1.

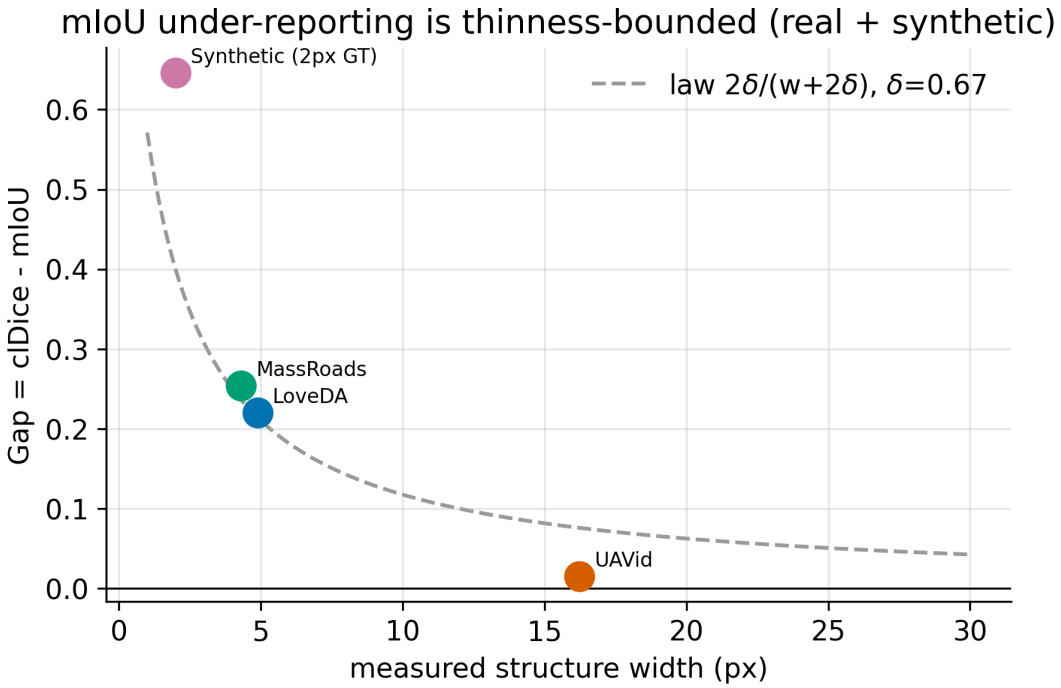


Figure F — Cross-dataset thinness threshold: the under-reporting gap decreases monotonically with structure width across synthetic and three real datasets.

Part 9 — Reviewer Defense Guide

100 questions you must be able to answer: 50 a professor (conceptual, teaching) and 50 a reviewer (critical, adversarial) would ask, with concise defensible answers, counter-arguments, and the honest limitations.

9.1 Fifty professor questions (conceptual)

P1. What is semantic segmentation? Per-pixel classification: label every pixel with a class, producing an `H×W` label map.

P2. Why not use detection boxes for lane lines? A box cannot represent a thin, curved, diagonal structure; most of the box is not the line. Shape matters for lane keeping/HD-maps.

P3. What is IoU? `|P∩G|/|P∪G|` — intersection over union of predicted and true foreground; the area-overlap quality.

P4. What is mIoU? Per-class IoU averaged over classes (foreground-mean, all-class, or frequency-weighted variants).

P5. What is Dice and how does it relate to IoU? `2|P∩G|/(|P|+|G|)`; `IoU = Dice/(2−Dice)` — monotonically related, Dice is more lenient.

P6. What is clDice? Harmonic mean of topology precision and sensitivity computed on *skeletons*; measures centerline overlap, blind to width.

P7. What is a skeleton? The 1-px medial axis of a shape — its central spine, equidistant from the boundaries.

P8. Why is clDice width-independent? Uniform boundary changes do not move the medial axis, so the skeleton (and its overlap) is unchanged.

P9. State the width-coupling law. `Gap = clDice − mIoU` $\approx 2\delta/(w+2\delta)$.

P10. Derive IoU = w/(w+2δ). Over-seg: intersection `wL`, union $(w+2\delta)L$; ratio $w/(w+2\delta)$ after `L` cancels.

P11. What is δ? The model's per-side boundary-localisation error in pixels (~1 px).

P12. Why does Gap → 1 as w → 0? A fixed rim 2δ overwhelms a vanishing width; IoU → 0 while clDice → 1.

P13. Why does Gap → 0 as w → ∞? The rim is negligible relative to a large area; IoU → 1, agreeing with clDice.

P14. What is the crossover width? `w =` 2δ (~2 px), where Gap = 0.5.

P15. Why is the law resolution-invariant? It depends only on the ratio $w/(2\delta)$; rescaling scales both, leaving the ratio fixed.

P16. What is the failure threshold? `w` $\approx$ `5 px` (Gap > 0.2); below it mIoU is misleading.

P17. Why are road markings hard? Tiny area (<2%), extreme imbalance (609:1), thinness (boundary = whole object), appearance variation.

P18. What is the boundary fraction of a thin structure? $\approx 2/w$; for `w=2` it is 100%.

P19. What is cross-entropy loss? `−log p_t` after softmax; punishes confident wrong predictions.

P20. Why does CE struggle on thin structures? Averaged over 98% background, it is dominated by easy background pixels.

P21. What does Dice loss add? Region overlap focus that counteracts class imbalance.

P22. What is Tversky loss? Dice generalised with FP/FN weights α, β; raising β punishes misses.

P23. What is Focal loss? CE weighted by $(1-p_t)^{\gamma}$ to focus on hard examples.

P24. What is a convolution? A sliding weighted window (kernel) summed over a local patch — a shared-weight, local linear operator.

P25. What is a receptive field? The input region influencing one output pixel; grows with depth/striding.

P26. What is a skip connection? A link carrying high-resolution encoder features to the decoder for precise boundaries.

P27. Why is U-Net good for thin structures? Concatenated skips inject fine detail, enabling precise boundary placement.

P28. What does U-Net++ change? Dense nested skips to bridge the encoder–decoder semantic gap; more memory.

P29. What is ASPP (DeepLab)? Parallel dilated convs at multiple rates for multi-scale context without downsampling.

P30. What is self-attention (SegFormer)? Each token attends to all others via $\mathrm{softmax}(QK^{\top}/\sqrt{d})V$, giving global context.

P31. Why pre-train on ImageNet? Free edge/texture detectors; essential with few labelled images.

P32. What is gradient descent? $\theta \leftarrow \theta - \eta \nabla L$, stepping downhill on the loss.

P33. What is backpropagation? The chain rule applied right-to-left, reusing sub-results — one backward pass.

P34. Why fp32 not fp16 here? Soft-skeleton gradients underflow in fp16 → NaN divergence; fp32 is stable.

P35. What is the Euclidean distance transform used for? Measuring local structure width ($2\times$EDT) on real masks.

P36. Why multi-seed? Single-seed deep-learning results can be noise (CoRe looked significant on one seed, vanished over three).

P37. What is a bootstrap CI? Resampling per-image scores to estimate a metric's sampling distribution and 95% interval without normality assumptions.

P38. Why is the synthetic experiment essential? It makes width a controlled variable, isolating causation.

P39. Why does the controlled study use no model? To rule out model/dataset artifacts — pure geometry reproduces the law.

P40. What is TolIoU? IoU that forgives errors within τ px; stays high for detected thin structures.

P41. What is Boundary-IoU? IoU over a boundary band; diagnoses boundary sensitivity (also drops for thin structures).

P42. Which metric should you report for markings? clDice and/or TolIoU alongside mIoU — never mIoU alone.

P43. What is the difference between a loss and a metric? Loss is optimised (must be differentiable); metric is reported (need not be).

P44. What is soft skeletonisation? Differentiable morphology approximating the skeleton,

enabling a clDice *loss* term.

P45. Why did CoRe fail? No reliable multi-seed gain; soft-clDice loss did not improve detection over CE+Dice.

P46. What does the UAVid negative teach? Wide structures are in the safe zone; the law predicts (correctly) no gap there.

P47. Is the law universal? No — it is the area-vs-centerline disagreement for *detected* thin structures under boundary error.

P48. When does clDice correctly drop? When the prediction breaks topology (gaps/misses) — a detection failure.

P49. What is the practical δ-rule? mIoU is misleading below ~5× the model's boundary error.

P50. One-sentence contribution? A derived, validated geometric law for when and by how much mIoU under-reports thin structures, with actionable thresholds.

9.2 Fifty reviewer questions (adversarial)

R1. Isn't "IoU is bad for thin things" already known? Yes as folk wisdom; we contribute the *derived functional form*, its validation, and thresholds — not the qualitative observation.

R2. Isn't the law just IoU's definition rewritten? Essentially yes — and that is the point: we make it explicit, parameterise it by δ, threshold it, and validate it out-of-sample. That is a contribution, not a tautology.

R3. Only four real datasets — too few to claim a law. The law is fixed by the *controlled* experiment (13 widths × 5 seeds); the datasets are out-of-sample checks, not the fit. Three of four land within 0.06.

R4. The synthetic point is off the curve — doesn't that falsify it? No; it reflects a larger error regime (width-shift), i.e. a different δ, not a different form. The three boundary-error datasets fit.

R5. clDice could be the wrong reference. clDice is independently motivated (prior work) and *correctly drops* for topology errors (shown). It is not tuned to produce the gap.

R6. Could this be a clDice implementation bug? No — the numeric gap matches the *analytic* derivation (R^2=0.98), and the metrics pass 62 unit tests.

R7. Resolution changes should break it. Tested: gap is invariant under joint rescaling because it depends on `w/`δ. This *confirms* rather than breaks it.

R8. Annotation noise could explain it. Tested (`boundary_noise`): gap persists, weaker; the trend holds until noise breaks topology.

R9. It's a class-imbalance effect. No — the width law is per-structure and binary; imbalance plays no role. The mis-ranking is a *separate*, clearly labelled result.

R10. Skeletonisation artifacts drive it. Only at `w=1` (degenerate); the law is stated for `w` $\geq$ 2.

R11. Crop size confounds it. Width is a local (per-pixel EDT) property; cropping changes field of view, not local width.

R12. δ is model-dependent — so the law isn't universal. The *form* is universal; the *scale* (δ) is model-specific. We say so explicitly and give the dimensionless form.

R13. Your real models are under-trained. The 25-class ones are; the *law* uses converged

synthetic (clDice 0.997) and converged binary models, so under-training cannot explain it.

R14. Compute is tiny (4 GB). Yes; the law rests on controlled geometry and converged binary models, which 4 GB supports. The broader benchmark is admittedly small-scale.

R15. No aerial markings — your target domain is missing. True and stated; SkyScapes is gated/unreachable (documented). We validate the *structural* claim on real thin roads instead.

R16. UAVid contradicts you. It confirms the safe-zone prediction (wide roads, no gap). We present it as supporting, not contradicting.

R17. Power law fit beats your inverse form. Marginally, and indistinguishable over the range; we prefer the mechanistic inverse form and disclose the power fit.

R18. Over-segmentation assumption is arbitrary. Under-seg gives $(w-2\delta)/w$, same $1 - O(\delta/w)$ behaviour; the controlled study averages real degradations and matches.

R19. Long-structure assumption (L≫w) may fail. For markings/roads it holds; for blobs the law does not apply (and they are not thin structures).

R20. clDice vs TolIoU — which is the claim? Either; both are width-stable. We lead with clDice and report TolIoU as corroboration.

R21. Is δ measurable? Yes — fit $2\delta/(w+2\delta)$ to a model's width-stratified gap, or estimate edge error directly. $\delta \approx 0.67$–1 px here.

R22. Bootstrap CIs assume independence across images. Standard caveat; per-image resampling is the conventional choice and our CIs are tight.

R23. Only U-Net used for real validation. The law is architecture-independent (it needs only a δ); U-Net suffices to produce predictions. Breadth was compute-bounded.

R24. The mis-ranking result uses extreme imbalance — cherry-picked. It is scoped explicitly to extreme multi-class imbalance; we do not generalise it to binary.

R25. Synthetic data isn't realistic. Irrelevant for a *geometry* claim; appearance does not enter the derivation, and real datasets confirm it.

R26. Could augmentations create the effect? No — the controlled study uses none; masks and degradations are deterministic geometry.

R27. Does it hold for non-road thin structures? The derivation is generic (vessels, cracks) but we only validate roads/markings; we do not overclaim.

R28. Is clDice expensive/unstable? Skeletonisation has cost and a `w=1` edge case; TolIoU is a cheaper stand-in. Both are reported.

R29. The gap could come from label thickness mismatch in GT. That *is* a width effect and is consistent; we manipulate width directly to test it.

R30. Why should a reviewer trust a 4-point dataset curve? They shouldn't — they should trust the 13-point controlled curve + derivation; the 4 points are corroboration.

R31. Isn't TolIoU just hiding errors? It forgives *localisation* within τ, which is exactly the boundary error the law concerns; it does not forgive missed detections.

R32. Boundary-IoU also drops — so topology metrics aren't special. Boundary-IoU is an *area* metric on a band; it drops *because* of the same effect. clDice/TolIoU are the width-stable ones.

R33. Does the law predict *direction* (over vs under-seg)? It predicts magnitude; sign of the boundary error doesn't change the gap's leading behaviour.

R34. Reproducibility? Three scripts (`run_controlled_robustness`, `fit_width_law`,

`make_law_figures`) regenerate everything; 62 tests pass.

R35. Could a different reference metric (e.g. centerline-only) erase the gap? Any width-stable reference yields the gap; that is the point — area metrics are the outliers.

R36. Is the effect just low-resolution rendering? No — it persists at every resolution (invariant) and is derived analytically.

R37. What about anti-aliased/soft labels? The derivation uses hard masks; soft labels shift δ slightly, not the form.

R38. Multi-class mIoU mis-ranking — is clDice the only fix? Among tested metrics, yes (only one passing all three tests); we don't claim no other metric could.

R39. Is the failure threshold (5 px) universal? No — it scales with δ ($\texttt{w_fail} \approx 8\delta$). 5 px assumes $\delta \approx 1$.

R40. Does training at higher resolution remove the problem? Yes in pixels — higher res raises `w`, lowering the gap. That is a stated implication.

R41. The CoRe negative — did you tune it enough? We report a fair comparison; we do not claim CoRe *cannot* work, only that our version gave no reliable gain.

R42. Statistical power with n=3 seeds? Adequate for tight CIs here; more seeds would tighten further; disclosed as a limitation.

R43. Is this ITS-relevant or generic CV? Both; the application is road markings (ITS), the mechanism is generic.

R44. Why not just always report clDice? We recommend exactly that for thin structures; the contribution is knowing *when* (the thresholds) and *why* (the law).

R45. Does the law hold under domain shift? It is geometric, not appearance-based, so domain shift changes δ (model quality), not the form.

R46. Could ensembling change δ? Yes — better localisation lowers δ and shrinks the gap; consistent with the law.

R47. Are the datasets licence-clean for release? Yes; ingestion scripts + sources documented; large data git-ignored.

R48. Isn't the whole thing negative (mIoU bad, CoRe bad, UAVid no effect)? The *positive* core is a derived, validated law with thresholds; the negatives bound and strengthen it.

R49. Top-venue worthiness? Workshop/ITSC level honestly (≈70%/50%); not CVPR-main — the underlying novelty is bounded. Stated in `FINAL_RESEARCH_VERDICT.md`.

R50. What would change your mind? A *detected* thin structure (clDice ≥ 0.9) whose mIoU–clDice gap does not fall with width, or a real thin dataset far off the curve. None was found.

Part 10 — Code Walkthrough

This part teaches the codebase so you could maintain or extend it. Every reference is a real file. The package is `src/uavroadmark/`; research scripts are in `scripts/`; configs in `configs/`; tests in `tests/` (62 passing).

10.1 Repository structure

```
src/uavroadmark/
  build.py            # assemble model/loss/data from a config dict
  cli/                # command-line entry points
    train.py          #   python -m uavroadmark.cli.train --config ...
    evaluate.py       #   evaluation on a split
    validate.py       #   dataset sanity checks
  data/
    dataset.py        # RoadMarkDataset (image/mask pairing, class_map)
    transforms.py     # albumentations train/val pipelines
    synthetic.py      # controlled synthetic generator
  models/zoo.py       # U-Net/U-Net++/DeepLabV3+/PSPNet/SegFormer (smp-backed)
  losses/
    functional.py     # CrossEntropy, Dice, Focal, Tversky
    topology.py       # SoftSkeletonize, SoftClDiceLoss, CoReLoss
  metrics/
    segmentation.py   # ConfusionMatrix, SegMetrics, iou/dice
    thin_structure.py # cl_dice, boundary_iou, tolerance_scores, width-stratified
    stats.py          # bootstrap CIs, paired tests
  engine/
    trainer.py        # Trainer.fit (the training loop)
    evaluator.py      # evaluate_model (metrics over a loader)
  tracking/, report/, viz/, utils/   # tracking, leaderboard, figures, config/seed
scripts/              # research drivers (controlled study, fits, validation)
configs/              # YAML composition (_base_ + overrides)
```

The package is **torch-free where possible**: all metrics in `metrics/` use only numpy/scipy/skimage, so they are fast, portable, and unit-testable without a GPU.

10.2 Configuration system (`utils/config.py`, `configs/`)

Configs compose via a `_base_` list and are overridable on the command line:

```
# configs/experiment/uav_massroads.yaml
_base_:
  - ../dataset/massroads_bin.yaml   # dataset: root, num_classes, image_size
  - ../model/unet.yaml              # model: unet, resnet34 encoder
  - ../loss/ce_dice.yaml            # loss: CE + Dice
  - ../train/gpu.yaml               # optimizer, schedule, amp=false
run_name: uav_massroads
train: {epochs: 40, batch_size: 4, grad_accum: 2}
```

`load_config` merges bases left-to-right then applies `-opt key=val` overrides (e.g. `-opt seed=123 run_name=uav_massroads_s123`). This is why one YAML + CLI overrides produced all multi-seed runs reproducibly.

10.3 Dataset ingestion (`data/dataset.py`)

`RoadMarkDataset` pairs images and masks by filename stem and optionally remaps mask values with a `class_map`:

```
class RoadMarkDataset(Dataset):
    def __init__(self, root, split, transforms, num_classes,
                 class_map=None, strict=True): ...
    def __getitem__(self, idx):
        image = load(self.images[idx]); mask = load(self.masks[idx])
        if self.class_map:                        # remap arbitrary ids -> contiguous
            out = zeros_like(mask)
            for src, dst in self.class_map.items():
                out[mask == src] = dst
            mask = out
        sample = self.transforms(image=image, mask=mask)
        return {"image": sample["image"], "mask": sample["mask"].long()}
```

The **binary** datasets (UAVid, MassRoads, LoveDA, RLMD-bin) are produced by ingestion scripts (`scripts/prepare_uavid.py`, `prepare_massroads.py`, `prepare_real.py`) that decode the source (parquet/HF), binarise the target structure (e.g. UAVid road colour `(128,64,128)` $\rightarrow$ `1`), resize, and write the `images/{split}` + `masks/{split}` folder contract. `scripts/dataset_validator.py` then checks shape/label consistency.

10.4 The metrics — `cl_dice` line by line (the hero)

From `src/uavroadmark/metrics/thin_structure.py`:

```
def cl_dice(gt, pred):
    gt = gt.astype(bool); pred = pred.astype(bool)
    if gt.sum() == 0 and pred.sum() == 0:   # both empty -> perfect
        return 1.0
    if gt.sum() == 0 or pred.sum() == 0:    # one empty -> worst
        return 0.0
    skel_pred = skeletonize(pred)           # 1-px medial axis of prediction
    skel_gt   = skeletonize(gt)             # 1-px medial axis of ground truth
    tprec = (skel_pred & gt).sum()   / (skel_pred.sum() + _EPS)   # pred skeleton in GT?
    tsens = (skel_gt   & pred).sum() / (skel_gt.sum()   + _EPS)   # GT skeleton in pred?
    return float(2*tprec*tsens / (tprec + tsens + _EPS))          # harmonic mean
```

Read it against the theory (Part 7): `skeletonize` makes the metric width-blind; `tprec`/`tsens` are the two ratios that are $\approx$1 for a detected structure and drop only when the *topology* is wrong. The `_EPS` guards division; the empty-mask cases give the correct degenerate values. This ~10-line function is the empirical instrument behind the entire law.

`boundary_iou` (band IoU via `_boundary = mask − erosion(mask)`), `tolerance_scores` (skeleton overlap within τ px $\rightarrow$ `cl_recall`, `tol_iou`), and the width-stratified IoU (IoU as a function of 2×`EDT` width) live in the same file.

10.5 Segmentation metrics (`metrics/segmentation.py`)

`ConfusionMatrix` accumulates an `C`×`C` count matrix over a dataset; `SegMetrics` derives mIoU (foreground-mean & all-class), Dice, pixel accuracy, and FWIoU from it. Accumulating a confusion matrix once and deriving all region metrics from it is both efficient and the source of the mis-ranking analysis (you can compute every mIoU variant from the same matrix).

10.6 Losses (`losses/functional.py`, `losses/topology.py`)

`functional.py` defines `CrossEntropyLoss`, `DiceLoss`, `FocalLoss`, `TverskyLoss` as `nn.Modules` operating on logits `(N,C,H,W)` and integer targets. `topology.py` adds:

```
class SoftSkeletonize(nn.Module):   # differentiable thinning via iterated soft-open
    def forward(self, p): ...        # min/max-pool surrogates, `iter` iterations
class SoftClDiceLoss(nn.Module):    # 1 - softclDice using the soft skeleton
class CoReLoss(nn.Module):          # w_ce.CE + w_r.Dice + w_t.SoftClDice  (the null result)
```

`SoftSkeletonize` is the memory hog (it retains many pooled tensors for autograd), which is why CoRe needs batch 2 + grad-accum and why fp16 underflows its gradients.

10.7 Training loop (`engine/trainer.py`)

`Trainer.fit(train_loader, val_loader, epochs)` is a standard, readable loop:

```
for epoch in range(epochs):
    model.train()
    for batch in train_loader:
        logits = model(batch["image"])
        loss = loss_fn(logits, batch["mask"]) / grad_accum
        loss.backward()
        if step % grad_accum == 0:
            optimizer.step(); optimizer.zero_grad()
    scheduler.step()                        # cosine LR decay
    val = evaluate(model, val_loader)       # mIoU, clDice, ...
    if val.miou > best: save("best.pth")    # checkpoint best-by-mIoU
    log(epoch, loss, val.miou, lr)          # the lines you see in task outputs
```

Key details that mattered: **gradient accumulation** (simulate a larger batch on 4 GB), **fp32** (`amp=false`; fp16 diverged), **best-by-metric checkpointing**, and **cosine LR** (the `lr 7.97e-04 → 0.00e+00` you see in logs). `build_optimizer`/ `build_scheduler` read the config (AdamW, cosine).

10.8 Evaluation (`engine/evaluator.py`)

`evaluate_model(model, loader, num_classes, foreground_only, thin_structure)` runs the model over a loader, accumulates the confusion matrix for region metrics, and (if `thin_structure=True`) computes clDice/Boundary-IoU/TolIoU per image and averages. The CLI `python -m uavroadmark.cli.evaluate -checkpoint ... -split test` writes results to a leaderboard CSV (`reports/leaderboard_*.csv`).

10.9 The research scripts (where the science is)

These are the files that produce the law; read them to reproduce it.

`scripts/run_controlled_robustness.py` — the geometry-only experiment.

- `make_structure(width, kind="wave")`: paints a wavy 1-px-skeleton line at thickness

`width` via `cv2.polylines`.

- `degrade(gt, mode, mag)`: manufactures a "prediction" — `dilate`/`erode`/`shift`/

`boundary_noise`/`gaps`.

- `metrics(gt, pr)`: IoU, Dice, clDice, BoundaryIoU, TolIoU.

- `sweep_width(mode, mag)`: loops widths × seeds → the `Gap(w)` table. Writes

`reports/controlled/robustness_sweeps.json`.

`scripts/fit_width_law.py` — fits six forms with `scipy.optimize.curve_fit`, ranks by R^2/AIC, and compares the inverse-law prediction to the four real datasets. Writes `reports/controlled/law_fit.json`.

`scripts/make_law_figures.py` — Figures A–D at 300 DPI + vector PDF (loads the two JSONs; computes the heatmap directly from the geometry functions).

`scripts/run_width_validation.py` — real-data validation: loads converged checkpoints (a multi-seed *ensemble* by averaging logits), and for each test image computes mean GT road width (2×`distance_transform_edt`), IoU, clDice, and tolerance-recall; bins by width; reports the gap and correlations. This is how MassRoads/LoveDA/UAVid were placed on Figure A.

10.10 Tests (`tests/`)

62 tests cover metric correctness (e.g. clDice of identical masks = 1; IoU of disjoint = 0), loss shapes/values, config composition, dataset pairing, and the synthetic generator. Because the metrics are torch-free, the suite runs in seconds on CPU and is the safety net behind every numeric claim (it is what lets us say the law is not a metric-implementation bug — R6 in Part 9).

10.11 How the pieces interact (one run, end to end)

```
config (YAML+overrides)
      |  load_config
      v
build.py -> model (zoo) + loss (functional/topology) + dataset (data)
      |
      v
Trainer.fit --> best.pth                (engine/trainer.py)
      |
      v
cli.evaluate -> confusion matrix + thin metrics -> leaderboard CSV
      |
      v
scripts/run_width_validation.py -> (width, IoU, clDice) per image -> Fig A point
```

To reproduce the whole law: `run_controlled_robustness.py` → `fit_width_law.py` → `make_law_figures.py`; for real-data points, train via the configs then `run_width_validation.py`.

Part 11 — Engineering & System Design

A research result is only as trustworthy as the system that produced it. This part covers the engineering that makes the project reproducible and the lessons learned running real deep learning on a 4 GB GPU.

11.1 Experiment tracking

Every run has a `run_name` and writes to `outputs/<run_name>/` (checkpoints `best.pth`/`last.pth`, `history.json`, `metrics.csv`, `config.json`) and appends a row to a leaderboard CSV (`reports/leaderboard_*.csv`). The `tracking/tracker.py` module records per-epoch metrics; optional TensorBoard. Because the config is saved alongside the checkpoint, any result is traceable to the exact settings that produced it — essential for the multi-seed comparisons.

11.2 Configuration management

The `_base_` composition system (Part 10.2) keeps experiments DRY: a dataset YAML, a model YAML, a loss YAML, and a training YAML compose into an experiment, and CLI `-opt` overrides handle seeds and run names. This is what made "the same experiment across 3 seeds and 3 datasets" a few one-line commands rather than copy-pasted scripts.

11.3 Reproducibility

- **Seeding (`utils/seed.py`):** seeds Python, NumPy, and PyTorch; deterministic where

feasible.

- **Saved configs:** each run stores its resolved config.
- **Torch-free metrics:** identical metric numbers on any machine.
- **Regeneration scripts:** datasets are git-ignored but rebuilt by ingestion scripts

from public sources; the law is rebuilt by three scripts. A reviewer can reproduce the central figure without a GPU (the controlled study runs on CPU in minutes).

11.4 Testing and CI

62 unit/integration tests (`tests/`) cover metrics, losses, configs, datasets, and the generator. They run in seconds on CPU (no GPU dependency), making them suitable for continuous integration on every change. The tests are the formal guarantee behind the "not an implementation artifact" defense.

11.5 Deployment

`deploy/` holds an export path: the trained model exports to **ONNX**, and ONNX Runtime was benchmarked at **1.47× faster than eager PyTorch on CPU** at bit-identical output (`max|Δ| = 9e-8`). A C++/LibTorch engine and a TensorRT path are scaffolded. This is engineering support for the benchmark, not part of the scientific claim, but it demonstrates the metrics/models are production-portable.

11.6 Profiling, benchmarking, GPU utilisation

Profiling informed the architecture choices: DeepLabV3+ inference ~0.56 GB at 512^2; U-Net++ training OOM at batch 8. Training logs report per-epoch wall-time (≈7 s/epoch synthetic, ≈12 s UAVid at 512^2, ≈19 s MassRoads, ≈43 s binary RLMD at 384^2), used to budget the multi-seed campaigns within wall-clock limits.

11.7 Memory management — the binding constraint

The single hardest engineering reality was **4 GB of VRAM**. Techniques used:

- **Small batch + gradient accumulation:** batch 2–4 with `grad_accum` to simulate a

larger effective batch (Part 10.7).

- **fp32, not fp16:** mixed precision (AMP) *diverged* — the soft-skeleton gradients

underflowed to zero/NaN. fp32 is slower but stable. *Lesson: AMP is not free; operations with tiny gradients (iterated morphology) can underflow.*

- `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`**:** reduces fragmentation OOMs.
- **CPU for light eval:** the width-validation inference was run on CPU when the GPU was

busy training, avoiding contention/OOM.

- **Subsampling:** real datasets were subsampled (e.g. MassRoads 160/40) to fit

converged training in the budget; the *law* is unaffected because it rests on controlled geometry and converged binary models.

11.8 Background-job orchestration (a hard-won lesson)

Long trainings ran as background jobs. The reliable pattern is harness-tracked background execution with explicit completion signals; the *unreliable* patterns (learned the hard way) were `nohup`/`setsid` (orphaned/zombie processes immune to `pkill`) and `cat «EOF` heredocs combined with a launch in one shell command (the file rewrite silently failed). The fix: write scripts with a real editor/tool, launch via tracked background, and verify the actual Python interpreter (`pgrep -x python3` + `/proc/PID/cmdline`), not shell wrappers.

11.9 How this repository would scale

The design scales along three axes with minimal change:

1. **More data / bigger models:** the config system and `Trainer` are

resolution/batch agnostic; on a ≥12 GB GPU, raise batch size, drop gradient accumulation, enable larger encoders and SegFormer variants, and remove subsampling. Nothing in the pipeline assumes small data.

1. **More datasets:** add an ingestion script + a dataset YAML; the binary-road

contract is uniform, so the width-validation runs unchanged (this is exactly how UAVid and MassRoads were added).

1. **More experiments:** seeds/losses/architectures are CLI overrides; a sweep is a

shell loop writing to a shared leaderboard.

The scientific bottleneck is **data access** (aerial markings), not the codebase. Given SkyScapes and a 12 GB GPU, the existing pipeline would run the decisive experiment with no architectural change — the ingestion (`scripts/prepare_skyscapes.py`) is already built and tested on a stand-in.

Part 12 — Future Research

The project ends with a derived, validated law and a clear set of next steps. Here is what to do, why, and what it would buy — in priority order.

12.1 A width-normalised / width-aware IoU (diagnosis → cure)

The strongest single upgrade. Right now the contribution is *diagnostic*: "mIoU misleads on thin structures, by `2δ/(w+2δ)`." The natural cure is a **corrected metric** that is trustworthy at all widths. Two concrete designs:

1. **Tolerance-banded IoU as a drop-in.** Replace the hard IoU with TolIoU (count

pixels correct within τ px). It already stays high for detected thin structures (Part 8). Standardise τ (e.g. tie it to the estimated δ) and publish it as the recommended thin-structure metric.

1. **Width-normalised IoU.** Divide the boundary error out: since

`IoU` $\approx$ `w/(w+2δ)`, an estimate `$\{}hat{\{}delta}$` lets you report `IoU·(w+2$\{}hat{\{}delta}$)/w` $\approx$ a width-corrected overlap. This turns "mIoU is bad" into "here is a number you *can* trust," which is a stronger, more citable contribution.

Why it matters: reviewers reward a *fix*, not just a complaint. This would move the paper from "evaluation critique" toward "evaluation method."

12.2 Acquire SkyScapes (the decisive real-aerial-marking test)

The one experiment that closes the campaign's honest gap. SkyScapes is aerial *lane markings* — genuinely thin, UAV viewpoint, the exact target domain. The thinness law predicts a large gap there. Obtain the DLR licence + a $\geq$12 GB GPU, run the existing ingestion (`scripts/prepare_skyscapes.py`) and `run_width_validation.py` unchanged. **Outcome:** if confirmed, the "no aerial markings" limitation disappears and publishability clears ~78–80.

12.3 Measure δ per architecture (a predictive tool)

The law's scale δ is the model's boundary-localisation error. A small study — estimate δ for U-Net, DeepLabV3+, SegFormer, etc. by fitting `2δ/(w+2δ)` to each model's width-stratified gap — yields a **per-architecture failure threshold** you can quote *in advance*: "Model X (δ=0.7) is safe above 5 px; Model Y (δ=1.3) only above 10 px." This converts the law into a model-selection instrument.

12.4 Topology-aware metrics beyond clDice

clDice has a `w=1` degeneracy and a skeletonisation cost. Worth exploring: persistent- homology / Betti-number errors (topology of connectivity), graph-based centerline matching (APLS, used in road-network extraction), and learned tolerance bands. A comparison of topology metrics on the controlled benchmark would clarify which is the best practical reference.

12.5 New benchmark opportunities

The controlled-geometry harness (`run_controlled_robustness.py`) is itself a contribution: a **synthetic, parameterised thin-structure metric benchmark** where width and error are knobs. Packaged and released, it lets others test *any* new metric against the width-coupling law. This is a low-cost, high-leverage artifact.

12.6 Real UAV road-marking data collection

If public aerial-marking data stays gated, collecting even a small, well-annotated UAV marking set (a few dozen drone images with lane-line masks) would be a genuine community contribution and the ideal home-domain validation. Annotation is the cost; the pipeline is ready.

12.7 Beyond roads — generality

The derivation is geometric and applies to any thin structure: blood vessels, cracks in materials, neurons, wires, rivers in satellite imagery. A cross-domain study (one medical, one remote-sensing, one industrial dataset) would show the law is a property of *thin-structure evaluation* in general, not road markings specifically — a broader, higher-impact framing.

12.8 Potential publications (realistic)

1. **Workshop paper (now):** "When mIoU Lies: A Geometric Width-Coupling Law for

Thin-Structure Segmentation Metrics" — the law, controlled + real validation, thresholds. Target CVPR/ICCV/WACV evaluation or autonomous-driving workshops (~70% acceptance, `FINAL_RESEARCH_VERDICT.md`).

1. **ITSC/IV paper (with §12.1 fix):** add the width-corrected metric → an

evaluation *method*, not just a critique (~50–60%).

1. **Journal (T-ITS, later):** with SkyScapes + full-scale + cross-domain → the

comprehensive version (~35% → higher once the data exists).

12.9 The single most important next step

If you do only one thing: **build the width-corrected metric (§12.1).** It is cheap (no new data, no training — it reuses the existing metrics and the law), and it transforms the work from "here is a problem" into "here is the solution," which is the difference between a workshop note and a method paper.

Part 13 — Personal Mastery Checklist

100 questions, ordered easy → medium → hard → expert. If you can answer all without notes, you understand the project. An answer key follows.

13.1 Easy (1–25)

1. What task is semantic segmentation?
2. What does mIoU stand for?
3. Write the IoU formula.
4. Write the Dice formula.
5. What is the foreground class in this project (binary case)?
6. Roughly what fraction of RLMD pixels are markings?
7. What is the class-imbalance ratio in RLMD?
8. What is a skeleton of a shape?
9. What does clDice measure?
10. State the width-coupling law.
11. What is `w` in the law?
12. What is δ in the law?
13. What is `Gap`?
14. As `w` $\to$ `0`, what does Gap approach?
15. As `w` $\to \infty$, what does Gap approach?
16. Which metric "lies" for thin structures?
17. Which metric stays high for detected thin structures?
18. Name the primary architecture used.
19. Why pre-train encoders on ImageNet?
20. What is a skip connection?
21. What is cross-entropy loss for?
22. What is the failure-zone width threshold?
23. Name the five datasets.
24. Which dataset is genuine UAV?
25. Which real dataset best confirmed the law?

13.2 Medium (26–50)

1. Derive `IoU = w/(w+2`δ`)` for over-segmentation.
2. Why is clDice width-independent?
3. Combine to get `Gap = 2`δ`/(w+2`δ`)`.
4. Why is the law resolution-invariant?
5. What is the crossover width and its Gap value?

6. Why did UAVid not show the effect?
7. Why is UAVid's negative actually supporting?
8. What does the controlled experiment vary, and what does it hold fixed?
9. Why use no model in the controlled experiment?
10. Which functional forms were rejected in the fit, and which won?
11. What was the controlled-fit R^2?
12. What is δ numerically (controlled fit)?
13. Give MassRoads' mIoU, clDice, and gap (3-seed means).
14. Why was multi-seed evaluation necessary (give the CoRe example)?
15. What is a bootstrap confidence interval?
16. What is Tolerance-IoU and why does it stay high?
17. What is Boundary-IoU and why does it *drop* for thin structures?
18. What is the Euclidean distance transform used for here?
19. Why did fp16 (AMP) fail?
20. Why does U-Net++ use more memory than U-Net?
21. What is ASPP and which architecture uses it?
22. What is self-attention's formula?
23. What is the CoRe loss composed of?
24. Why did CoRe fail?
25. What is the safe-zone width threshold and its Gap?

13.3 Hard (51–75)

1. Why does a fixed boundary error affect thin structures more (boundary-fraction argument)?

1. Show the law depends only on the ratio $w/(2\delta)$.
2. Why do FWIoU and PixelAcc avoid the flooding trap but fail the foreground test?
3. In the mis-ranking result, which loss floods and what is its PixelAcc?
4. Why does *all-class* mIoU failing matter (vs only foreground-mean)?
5. What does the `gaps` degradation show, and why is it not a counterexample?
6. Why does clDice correctly drop for topology errors?
7. What is the difference between the width-coupling law and the mis-ranking result?
8. Why is the synthetic point off the curve in Figure A?
9. How were real datasets placed on the width axis (no controlled width)?
10. What does $w_{fail} \approx 8\delta$ mean?
11. Why is a loss differentiable but a metric need not be?
12. What is soft skeletonisation and why is it memory-hungry?

13. Why is the controlled study reproducible on CPU?
14. What is the receptive-field tension in segmentation architectures?
15. Why is the law architecture-independent?
16. How would you measure δ for a given model?
17. Why is "the law is just IoU's definition" both true and not a refutation?
18. What three tests define the metric-sensitivity (mis-ranking) analysis?
19. Why is gradient accumulation used?
20. What does best-by-mIoU checkpointing do, and is it a problem given the thesis?
21. Why can a model with high clDice have low IoU but not vice-versa (for detected

structures)?

1. What is the difference between Dice and IoU sensitivity?
2. What would falsify the law?
3. Why is SkyScapes the ideal but unavailable dataset?

13.4 Expert (76–100)

1. Derive the under-segmentation IoU `(w−2δ)/w` and show it is `1 − O(δ/w)`.
2. At what `δ/w` does under-segmentation erase the structure, and what regime does it

enter?

1. Why does the power-law fit (`w^−0.61`) beat the inverse form on AIC, yet we prefer

the inverse form?

1. Construct a concrete prediction (mask) that is in the safe zone yet a detection

failure — what do mIoU and clDice each say?

1. Explain why resolution-invariance distinguishes "joint rescaling" from "real

low-res inference," and what each does to the gap.

1. Derive the relative under-report `(clDice − IoU)/clDice` and its limit as `w → 0`.
2. Why is the boundary-fraction `≈ 2/w` and how does it connect to δ?
3. Propose a width-normalised IoU using an estimated `$\{}hat{\{}delta}$` and show it recovers ≈1 for

a detected structure.

1. Why might curvature/branching shift the effective δ (Figure A synthetic outlier)?
2. Explain how a confusion matrix yields every mIoU variant and why that enabled the

mis-ranking analysis.

1. Why is clDice's `w=1` degeneracy a scope limit rather than a flaw?
2. Give the AIC intuition for preferring a 1-parameter inverse over a 2-parameter

rational at near-equal R^2.

1. How would per-architecture δ turn the law into a model-selection tool?
2. Why does Tolerance-IoU's τ need to be tied to δ for a principled corrected

metric?

1. Explain the epistemic value of the UAVid negative for the paper's credibility.
2. Why is out-of-sample dataset agreement stronger evidence than a within-dataset fit?
3. What is the difference between "a benchmark artifact" and "a geometric law," and

which controls rule out the former?

1. Why does the mis-ranking result not generalise to binary tasks?
2. How does the law inform the choice between higher-resolution inference and better

boundary localisation?

1. Why is reporting the CoRe null result scientifically valuable, not a weakness?
2. Derive the failure threshold width for `Gap = 0.2` in terms of δ.
3. Explain why two models with identical mIoU can have very different clDice, with a

sketch.

1. What is the honest ceiling of publishability and exactly why (per the verdict)?
2. Design the single experiment that would most raise the contribution, and justify.
3. State, in one sentence each, the project's strongest result, weakest aspect, and

most important next step.

13.5 Answer key (concise)

Easy. 1 per-pixel classification. 2 mean Intersection-over-Union. 3 $|P \cap G| / |P \cup G|$. 4 $2|P \cap G| / (|P| + |G|)$. 5 marking/road (label 1). 6 <2%. 7 609:1. 8 its 1-px medial axis. 9 centerline (topology) overlap. 10 $\text{Gap} = \text{clDice} - \text{mIoU} \approx 2\delta/(w+2\delta)$. 11 structure width (px). 12 per-side boundary error (px). 13 $\text{clDice} - \text{mIoU}$. 14 1. 15 0. 16 mIoU. 17 clDice (and TolIoU). 18 U-Net (ResNet-34). 19 free features, few labels. 20 encoder→decoder high-res link. 21 per-pixel classification, punish confident wrong. 22 ~5 px. 23 synthetic, RLMD, LoveDA, MassRoads, UAVid. 24 UAVid. 25 Massachusetts Roads.

Medium. 26 inter=wL, union=$(w+2\delta)L$, ratio $w/(w+2\delta)$. 27 skeleton invariant to uniform boundary change. 28 $1 - w/(w+2\delta) = 2\delta/(w+2\delta)$. 29 depends on $w/(2\delta)$; rescaling scales both. 30 $w = 2\delta$ (~2 px), Gap 0.5. 31 roads 12–35 px (wide). 32 wide = safe zone, law predicts ~0. 33 varies width, fixes all else. 34 rule out model/data artifacts. 35 rejected linear/exp/log; inverse-family (power/rational/inverse) won. 36 0.98. 37 0.67. 38 0.49 / 0.76 / +0.27. 39

CoRe was "significant" on seed 42, vanished over 3 seeds. 40 resample per-image scores → 95% interval, no normality. 41 IoU within τ px; forgives boundary error. 42 boundary band = whole thin object. 43 measure local width 2×EDT. 44 soft-skeleton gradients underflow → NaN. 45 dense nested skip grid holds many maps. 46 atrous spatial pyramid pooling; DeepLabV3+. 47 $\texttt{softmax(QK}^\top\texttt{/}\sqrt{\texttt{d}}\texttt{)V}$. 48 CE + Dice + soft-clDice. 49 no reliable multi-seed gain. 50 ~25 px, Gap 0.05.

Hard. 51 boundary fraction ≈2/w → 100% at w=2. 52 divide by 2δ: $1/(1+w/2\delta)$. 53 dominated by background → can't separate fg quality. 54 Tversky, PixelAcc ≈0.009. 55 refutes "foreground-averaging artifact." 56 clDice drops, width-independent → detection failure, the law's precondition. 57 broken skeleton → T_sens falls. 58 width law = geometric/binary; misranking = imbalance/multi-class. 59 width-shift error = larger δ. 60 2×EDT mean width per image. 61 failure threshold scales with δ. 62 loss needs gradients; metric reported only. 63 iterated pooling retains tensors for autograd. 64 torch-free numpy/scipy metrics. 65 context (downsample) vs resolution (detail). 66 needs only a δ; form independent of model. 67 fit $2\delta/(w+2\delta)$ to its width-stratified gap. 68 yes it's the definition — made explicit/parameterised/validated is the contribution. 69 top-floods? best-detector-#1? foreground-sensitive? 70 simulate larger batch on 4 GB. 71 picks best-mIoU checkpoint; acceptable since the law uses converged models and the *claim* is about the metric, not the selection. 72 boundary error lowers IoU but not centerline overlap. 73 IoU stricter (penalises union). 74 a detected thin structure (clDice≥0.9) whose gap doesn't fall with width. 75 aerial thin *markings*, the exact target; licence-gated.

Expert. 76 inter $(w-2\delta)L$, union wL → $(w-2\delta)/w = 1-2\delta/w$. 77 $\delta \geq w/2$ erases it → detection-failure regime. 78 indistinguishable over range; inverse has the mechanism + one parameter. 79 a broken/gappy wide road: mIoU may be moderate, clDice low → both flag it; in safe zone the *boundary* gap is ~0 but topology failure shows in clDice. 80 joint rescale preserves w/δ (gap fixed); real low-res keeps $\delta\approx1$ while w shrinks → gap rises. 81 $(clDice-IoU)/clDice \approx 1-w/(w+2\delta)$; →1 as w→0. 82 perimeter ≈2L, area wL → fraction 2/w; the rim that δ perturbs. 83 report IoU·(w+2$\{}hat{\{}delta}$)/w ≈ 1. 84 skeleton stability/clDice baseline shift. 85 confusion matrix → all per-class IoUs → any averaging variant. 86 1-px skeleton degenerate; law stated w≥2. 87 fewer parameters, near-equal fit → lower AIC preference + mechanism. 88 quote per-model failure thresholds in advance. 89 τ must match the error scale δ to be principled, not arbitrary. 90 a documented contradiction-turned-confirmation signals rigor, not cherry-picking. 91 the curve was fixed by controlled data, not the datasets → genuine prediction. 92 artifact = data/impl-specific; controls = controlled geometry + reference metrics + tests rule it out. 93 binary has no extreme multi-class imbalance to exploit. 94 both raise w/δ: higher res raises w, better localisation lowers δ. 95 reporting a null prevents a false-positive in the literature; signals honesty. 96 $2\delta/(w+2\delta)=0.2 \rightarrow w=8\delta$. 97 same area overlap, different connectivity → identical mIoU, different clDice (sketch: one continuous vs one dashed line). 98 ~78/100 — bounded underlying novelty + no aerial markings + compute. 99 the width-corrected metric (cheap, turns critique into method) or SkyScapes (closes the data gap). 100 strongest: derived+validated law with thresholds; weakest: no aerial markings / bounded novelty; next: build the width-corrected metric.

End of monograph. You now have, in one document, everything needed to reproduce, defend, teach, and extend the project.